```
// AmbientWeather.cs
// FluxVerse P-15 r13 (split r14): ambient cycle pure logic cores (24h four-tier color
// wheel + weather FX rules + recycled drop field simulation).
// CEO order 18:45: the city joins real 24h time and real Shanghai weather (M1 acceptance
// item, TECH P-15 expansion). Data source: world/world-state.json, READ-ONLY poll every
// ~10s (perceptor scan owns all writes; the engine never writes world/ or any sibling repo).
// Flat reality keys consumed: city_day_phase, beijing_hhmm, weather_kind, weather_code,
// weather_wind_ms. Pure 2D only. Tier bounds mirror probes/clock.ps1 exactly (probe =
// single source of truth): dawn 5-8, day 9-16, dusk 17-19, night else. Palettes follow
// DESIGN section 9: dusk = warm purple-gold per art-target-dusk.png; night = deep
// blue-black (city lights carry the electric cyan); pink/purple only ever ambient sky
// tint, never a functional light. Alert rule mirrors probes/weather.ps1 exactly:
// wind &gt;= 17.2 m/s (gale family) or heavy WMO codes 65,67,75,77,82,95,96,99 -&gt; city-wide
// red alert band (functional red, five-color law).
// SEPARATE FILE LAW (r14): the MonoBehaviour adapter CityAmbient now lives in
// CityAmbient.cs (a component class must match its .cs file name or the saved scene
// reference dies across editor sessions); this file keeps ONLY headless-testable cores.
using System;
using UnityEngine;
namespace FluxVerse
{
    public enum AmbientTier { Dawn, Day, Dusk, Night }
    public enum WeatherMode { None, Rain, Snow }
    public struct AmbientPalette
    {
        public Color skyTop, skyBottom, tint, camBg;
        public float tintAlpha;
    }
    // pure logic: hour -&gt; tier -&gt; palette (headless-testable)
    public static class AmbientWheel
    {
        public static AmbientTier TierForHour(int h)   // bounds = probes/clock.ps1 (r217: dawn 5-7, day 8-16, dusk 17-1
        {
            if (h &gt;= 5 &amp;&amp; h &lt; 8) return AmbientTier.Dawn;
            if (h &gt;= 8 &amp;&amp; h &lt; 17) return AmbientTier.Day;
            if (h &gt;= 17 &amp;&amp; h &lt; 20) return AmbientTier.Dusk;
            return AmbientTier.Night;
        }
        public static AmbientTier TierFromName(string name)
        {
            if (!string.IsNullOrEmpty(name))
            {
                if (name == "dawn") return AmbientTier.Dawn;
                if (name == "day") return AmbientTier.Day;
                if (name == "dusk") return AmbientTier.Dusk;
                if (name == "night") return AmbientTier.Night;
            }
            return TierForHour(DateTime.Now.Hour);   // bootstrap when state absent (host runs UTC+8)
        }
        public static AmbientPalette PaletteFor(AmbientTier t)
```

```
    {
        AmbientPalette p = new AmbientPalette();
        switch (t)
        {
            case AmbientTier.Dawn:   // cool blue zenith over warm pink horizon
                p.skyTop = new Color(0.42f, 0.52f, 0.72f);
                p.skyBottom = new Color(0.98f, 0.68f, 0.52f);
                p.tint = new Color(1f, 0.72f, 0.55f); p.tintAlpha = 0.08f;
                break;
            case AmbientTier.Day:    // clean daylight, city tiles at full color
                p.skyTop = new Color(0.45f, 0.66f, 0.92f);
                p.skyBottom = new Color(0.78f, 0.87f, 0.96f);
                p.tint = Color.white; p.tintAlpha = 0f;
                break;
            case AmbientTier.Dusk:   // art-target-dusk: purple zenith, warm gold horizon
                p.skyTop = new Color(0.36f, 0.22f, 0.48f);
                p.skyBottom = new Color(0.98f, 0.58f, 0.32f);
                p.tint = new Color(1f, 0.62f, 0.42f); p.tintAlpha = 0.22f;
                break;
            default:                 // night: deep blue-black, cyan comes from city lights
                p.skyTop = new Color(0.015f, 0.02f, 0.06f);
                p.skyBottom = new Color(0.05f, 0.09f, 0.20f);
                p.tint = new Color(0.04f, 0.09f, 0.24f); p.tintAlpha = 0.45f;
                break;
        }
        p.camBg = p.skyTop;
        return p;
    }
}
// r156 (P-20260925-09 D1): tier-flip palette blend core. The wheel's hard cut was the
// "feel" defect (r153 finding: ApplyAmbient -&gt; PaletteFor = instant switch, zero lerp).
// The PLAY path (CityAmbient.TransitionAmbient) eases the five-value family
// (skyTop / skyBottom / tint rgb / tintAlpha / camBg) over BlendSeconds with the r14
// zero-end-velocity smoothstep; ApplyAmbient stays the instant settle primitive (boot
// build + the 53 proof call sites + tier-gated families). One Advance core serves
// play (Update) and proof (StepAmbient pumps) - the CameraRig law.
public class AmbientBlend
{
    public const float BlendSeconds = 2.5f;   // spec band 2-4s (r153 D1)
    AmbientPalette from, to;
    float elapsed, seconds = BlendSeconds;
    public bool Active { get; private set; }
    public void Begin(AmbientPalette f, AmbientPalette t, float dur)
    {
        from = f; to = t; elapsed = 0f;
        seconds = Mathf.Max(0.0001f, dur);
        Active = true;
    }
    public void End() { Active = false; }
    // one step; returns the blended palette, snapping to the target at completion
```

```
    public AmbientPalette Advance(float dt)
    {
        if (!Active) return to;
        elapsed += dt;
        float k = Mathf.Clamp01(elapsed / seconds);
        if (k &gt;= 1f) { Active = false; return to; }
        float s = RigMath.EaseInOut(k);   // r14 easing: zero velocity at both ends
        AmbientPalette p = new AmbientPalette();
        p.skyTop = Color.Lerp(from.skyTop, to.skyTop, s);
        p.skyBottom = Color.Lerp(from.skyBottom, to.skyBottom, s);
        p.tint = Color.Lerp(from.tint, to.tint, s);
        p.tintAlpha = Mathf.Lerp(from.tintAlpha, to.tintAlpha, s);
        p.camBg = Color.Lerp(from.camBg, to.camBg, s);
        return p;
    }
    public float Progress01 { get { return Mathf.Clamp01(elapsed / seconds); } }
}
// pure logic: probe weather values -&gt; FX mode + alert flag (headless-testable)
public static class WeatherRules
{
    public static readonly int[] HeavyWmo = { 65, 67, 75, 77, 82, 95, 96, 99 };   // = weather.ps1
    public static WeatherMode ModeForKind(string kind)
    {
        if (kind == "rain" || kind == "thunder") return WeatherMode.Rain;   // thunderstorm rains
        if (kind == "snow") return WeatherMode.Snow;
        return WeatherMode.None;   // clear/cloud/fog/other: no particle field
    }
    public static bool IsAlert(float windMs, int wmoCode)
    {
        if (windMs &gt;= 17.2f) return true;
        foreach (int c in HeavyWmo) if (wmoCode == c) return true;
        return false;
    }
}
// pure simulation: recycled drop field (positions/speeds only, no GameObjects -&gt; sandbox-testable)
public class WeatherField
{
    public const float XHalf = 36f, YTop = 23f, YBottom = -23f;   // covers ortho-20 view + margin
    public readonly int Count;
    public WeatherMode Mode { get; private set; }
    public float WindX { get; private set; }
    readonly float[] x, y, v, phase;
    readonly System.Random rng;
    public WeatherField(int count, int seed)
    {
        Count = count;
        x = new float[count]; y = new float[count]; v = new float[count]; phase = new float[count];
        rng = new System.Random(seed);
    }
    public void Configure(WeatherMode mode, float windMs)
```

```
    {
        Mode = mode;
        WindX = Mathf.Clamp(windMs / 25f, -1.5f, 1.5f) * 12f;   // wind leans the fall
        for (int i = 0; i &lt; Count; i++) Respawn(i, true);
    }
    void Respawn(int i, bool anywhere)
    {
        x[i] = (float)(rng.NextDouble() * 2.0 - 1.0) * XHalf;
        y[i] = anywhere ? (float)(rng.NextDouble() * 2.0 - 1.0) * YTop
                        : YTop + (float)rng.NextDouble() * 3f;
        v[i] = Mode == WeatherMode.Rain ? 30f + (float)rng.NextDouble() * 18f
                                        : 4.5f + (float)rng.NextDouble() * 3.5f;
        phase[i] = (float)rng.NextDouble() * 6.283f;
    }
    public int Step(float dt)   // returns recycles this step (drop crossed the floor)
    {
        int recycled = 0;
        for (int i = 0; i &lt; Count; i++)
        {
            y[i] -= v[i] * dt;
            x[i] += (Mode == WeatherMode.Snow ? Mathf.Sin(phase[i] + y[i] * 0.35f) * 1.2f : WindX) * dt;
            if (y[i] &lt; YBottom) { Respawn(i, false); recycled++; }
        }
        return recycled;
    }
    public Vector3 Pos(int i) { return new Vector3(x[i], y[i], 0f); }
    public float Y(int i) { return y[i]; }
}
// P-28 item 2 (r34): parallax skyline pure rules. Source pack = parallax-skyline
// (OGA-BY 3.0, ledger in City/Assets/ArtPacks/ARTPACKS-LEDGER.md). The r29 style
// gate passed the pack conditionally with five laws, all enforced here:
//   1 point filter + INTEGER scale only (x2 at PPU 16 -&gt; texel 0.125u),
//   2 fog desaturation tint into the anchor pink-purple fog band (per tier),
//   3 only the 2 farthest pale silhouette layers (near-black layers rejected),
//   4 no seamless-loop claim -&gt; the single quad is never tiled/repeated, so its
//     width must cover every camera position (Covers gate below),
//   5 five-color law untouched: silhouettes are AMBIENT scenery (pink/purple
//     is a legal ambient sky color per DESIGN section 9, never a functional light).
// Pink/purple stays ambient-only; geometry mirrors the measured pack content rows
// (r33 survey + r34 PS re-measure: layer-2 rows 61..234, layer-3 rows 79..323).
public static class SkylineRules
{
    public const float FollowFactor = 0.9f;          // parallax: tracks 0.9 x camera
    public const int FarOrder = -9, NearOrder = -8;  // between AmbientSky -10 and tilemaps 0..4
    public const float Scale = 2f;                    // law 1: integer multiple only
    public const float PPU = 16f;
    public const float Texel = Scale / PPU;           // 0.125u per source texel
    public const float SourceW = 576f, SourceH = 324f;
    public const int FarTopRow = 61, FarBotRow = 234;      // layer-2 measured content rows
    public const int NearTopRow = 79, NearBotRow = 323;    // layer-3 measured content rows
```

```
    public const float FarContentTopY = 19f;         // far spires peek 4u above the +15 roofline
    public const float NearContentTopY = 17.5f;      // near skyline plateaus lower (depth stack)
    public const float RoofY = 15f;                 // painted band top edge (r22 survey)
    // quad center Y so the content (first opaque row) lands exactly at contentTopY
    public static float QuadCenterY(int topRow, float contentTopY)
    {
        float quadTop = contentTopY + topRow * Texel;
        return quadTop - (SourceH * Texel) * 0.5f;
    }
    // law 4 gate: non-looping quad must span the view at every camera position
    public static bool Covers(float camX, float halfView)
    {
        float half = SourceW * Texel * 0.5f;   // 36u
        float cx = camX * FollowFactor;
        return (cx - half) &lt;= (camX - halfView) &amp;&amp; (camX + halfView) &lt;= (cx + half);
    }
    // law 2: per-tier fog multiplier on the pack's native rose (240,147,161)/255.
    // Multiplication compresses the r-g spread (desaturation) and can only darken,
    // which is exactly "recede into fog". r208 (T-FV-122 S4, batch-2 order item 4
    // "far buildings lean blue, more fog"): dusk re-authors the mauve band into a
    // BLUE-PURPLE HAZE - the product on the rose base goes b-r +0.11 (the blue lean
    // of the order word) and the r-g spread compresses to ~0.44x native (was
    // ~0.69x = the "more fog" half). Single source law: LightFxRules.FogWashColor()
    // rides this same constant (r181), re-pinned in lightfx-manifest v0.4.
    public static Color FogFar(AmbientTier t)
    {
        switch (t)
        {
            case AmbientTier.Dawn: return new Color(0.86f, 0.72f, 0.80f);  // dusty rose haze
            case AmbientTier.Day:  return new Color(0.85f, 0.88f, 0.98f);  // pale cool haze
            case AmbientTier.Dusk: return new Color(0.50f, 0.54f, 0.92f);  // r208 blue-purple haze band
            default:               return new Color(0.10f, 0.10f, 0.18f);  // dark blue mass
        }
    }
    // near layer = slightly heavier fog-darkening (nearer silhouette reads as the
    // denser mass; classic depth cue for dark aerial perspective)
    public static Color FogNear(AmbientTier t) { return FogFar(t) * 0.85f; }
}
// r206 (T-FV-122 S2b, batch-2 order 09-26 ~15:20 item 1): the tint band
// splits into THREE blocks. The r13 single quad covered -16..+15 with one
// alpha; in LINEAR color space (r205 forensics: ProjectSettings
// m_ActiveColorSpace=1) the warm dusk tint (1,0.62,0.42 @ a0.22) over the
// dark blue-purple water reads ROSE (measured hue 319 vs the order door
// [220,280] - the "pink-brick texture" critique root). The batch-2 order
// text ("water semi-transparent blue-purple") is the later CEO word and
// overturns the r13 coverage domain (r202 sec.6a compliant-change family).
// LAW: the RIVER block mirrors the water world rect EXACTLY (WaterFxRules
// cells x -50..50 rows -3..2) and carries alpha ONLY AT NIGHT (the night
// tint is blue-family: water stays in-door, r205 measured 233/67; dusk and
// dawn read the authored blue-purple, r205 probe preview 239/92). The two
```

```
    // CITY blocks keep the four-tier wheel law untouched (r51 pilot bottom
    // -16 / painted top +15, 92u family width). Seams are flush by constant:
    // S top == river bottom == -3, river top == N bottom == +3.
    public static class TintBandRules
    {
        public const string NameS = "AmbientTintS", NameN = "AmbientTintN", NameRiver = "AmbientTintRiver";
        public const int Order = 8;                 // the legacy tint family slot (above street 7, below band 9)
        // south city block (paved band below the water)
        public const float CityW = 92f;              // legacy family width: x -46..+46 (L1 |camX|&lt;=30 law consumer)
        public const float Sy0 = -16f, Sy1 = -3f;    // r51 pilot bottom law / flush with the water south edge
        public const float SyH = Sy1 - Sy0;          // 13
        public const float SyCy = (Sy0 + Sy1) * 0.5f;
        // north city block (painted band up to the r51 +15 top)
        public const float Ny0 = 3f, Ny1 = 15f;
        public const float NyH = Ny1 - Ny0;          // 12
        public const float NyCy = (Ny0 + Ny1) * 0.5f;
        // river block = the water world rect exactly (single source = WaterFxRules)
        public const float Rx0 = WaterFxRules.CellX0;                // -50
        public const float Rx1 = WaterFxRules.CellX1 + 1f;           // +51 (cells x -50..50)
        public const float Ry0 = WaterFxRules.RowY0;                 // -3
        public const float Ry1 = WaterFxRules.RowY1 + 1f;             // +3 (rows -3..2)
        public const float RiverW = Rx1 - Rx0;                        // 101
        public const float RiverH = Ry1 - Ry0;                        // 6
        public const float RiverCx = (Rx0 + Rx1) * 0.5f;              // +0.5 (MakeQuad centers at x0)
        public const float RiverCy = (Ry0 + Ry1) * 0.5f;              // 0
        // the S2b river alpha law: night carries the wheel alpha, dusk/dawn/day exempt
        public static float RiverAlphaFor(AmbientTier t)
        {
            return t == AmbientTier.Night ? AmbientWheel.PaletteFor(t).tintAlpha : 0f;
        }
    }
    // scene adapter CityAmbient lives in CityAmbient.cs (SEPARATE FILE LAW, r14)
}
// AudioRouter.cs
// FluxVerse P-27 r25 (first wire) -&gt; r30 (rows batch 2): event type -&gt; clip pure core.
// R-20260923-audio-assets.md section 1.1 mapping table. v0 (r25) = CEO_ORDER only;
// r30 widens to every row whose event type is ACTUALLY EMITTED by the live stream
// (registry + world-events.jsonl type census) AND whose clip sits in the pack:
//   COMMIT-&gt;Laser_00, TASK_CLAIM-&gt;Robot_Activated_00, MARKET_OPEN/CLOSE-&gt;bells,
//   WEATHER_ALERT-&gt;s0_red_alert, RESIDENT_SAY-&gt;Robot_Talk_00.
// Rows with no live event source yet stay OUT (honesty law: sound anchors a real
// event, never decoration): OS_TICK/GATE_*/TRANSFER/MACHINE_* are P-12 pending.
// Volume canon (P-27 item 3): signature layer 1.0, event-SFX layer 0.8.
// r31 (P-27 item 2): FluxAmbientBed appended - four-tier BGM by city_day_phase +
// weather layer by weather_kind + always-on city noise bed (same volume canon family).
// Pure logic, zero GameObject/AudioSource: headless-testable; the CityEventRouter
// adapter owns the real player delegate. Unmapped type = silent, never a fallback
// jingle. All comments ASCII. No 3D.
using System;
using System.Collections.Generic;
```

```
namespace FluxVerse
{
    public class FluxAudioRouter
    {
        public const float SignatureVolume = 1.0f;   // P-27 item 3 mixing canon: signature 1.0
        public const float SfxVolume = 0.8f;         // P-27 item 3 mixing canon: event SFX 0.8
        public const string CeoOrderClip     = "Assets/Audio/signature/ceo_order_pulse.wav";
        public const string CommitClip       = "Assets/Audio/sfx_scifi/Laser_00.mp3";
        public const string TaskClaimClip    = "Assets/Audio/sfx_scifi/Robot_Activated_00.mp3";
        public const string MarketOpenClip    = "Assets/Audio/signature/market_bell_open.wav";
        public const string MarketCloseClip  = "Assets/Audio/signature/market_bell_close.wav";
        public const string WeatherAlertClip = "Assets/Audio/signature/s0_red_alert.wav";
        public const string ResidentTalkClip = "Assets/Audio/sfx_scifi/Robot_Talk_00.mp3";
        class Row
        {
            public readonly string Path;
            public readonly float Vol;
            public Row(string p, float v) { Path = p; Vol = v; }
        }
        // R- 1.1 mapping table, one row per live event type (single source of truth:
        // FluxEventRouter's cheap gate reads MappedTypes so gate and map can never drift).
        static readonly Dictionary&lt;string, Row&gt; map = new Dictionary&lt;string, Row&gt;
        {
            { "CEO_ORDER",     new Row(CeoOrderClip,     SignatureVolume) },
            { "COMMIT",        new Row(CommitClip,       SfxVolume) },
            { "TASK_CLAIM",    new Row(TaskClaimClip,    SfxVolume) },
            { "MARKET_OPEN",   new Row(MarketOpenClip,   SignatureVolume) },
            { "MARKET_CLOSE",  new Row(MarketCloseClip,  SignatureVolume) },
            { "WEATHER_ALERT", new Row(WeatherAlertClip, SignatureVolume) },
            { "RESIDENT_SAY",  new Row(ResidentTalkClip, SfxVolume) },
        };
        readonly Action&lt;string, float&gt; player;   // (assetPath, volume)
        public int Plays { get; private set; }
        public FluxAudioRouter(Action&lt;string, float&gt; player)
        {
            this.player = player;
        }
        public static ICollection&lt;string&gt; MappedTypes { get { return map.Keys; } }
        public static string SoundFor(string type)
        {
            Row r;
            return map.TryGetValue(type, out r) ? r.Path : null;
        }
        public static float VolumeFor(string type)
        {
            Row r;
            return map.TryGetValue(type, out r) ? r.Vol : 0f;
        }
        // wired as FluxEventRouter.EventSink; unmapped = silent (honesty law)
        public void OnEvent(FluxEvent ev)
```

```
        {
            if (ev == null) return;
            Row r;
            if (!map.TryGetValue(ev.type, out r)) return;
            if (player != null) player(r.Path, r.Vol);
            Plays++;
        }
    }
    // r31 (P-27 item 2): ambient bed pure core - four-tier BGM by city_day_phase +
    // weather layer + always-on city noise. Mapping = R-20260923-audio-assets.md
    // section 1.2 (semantic inference; listening calibration = P-27 item 6 v2).
    // Volume canon (P-27 item 3): BGM 0.35 (never covers city information),
    // ambient/environment layer 0.5. Tier tracks: dawn=calm_synthwave, day=synth_wave_0,
    // dusk=tt_caves (smallest of the cyber pool; pool rotation/crossfade = v2),
    // night=midnight_drive. Weather: rain=rain_loop_2, gale/severe alert enhances to
    // rain_loop_3; snow = silence (no snow asset in the pack - honesty law, never a
    // substitute jingle); dry wind &gt;= 10.8 m/s (Beaufort 6 strong-breeze family) =
    // wind2 layer. Headless-testable; the CityAmbientAudio adapter owns the players.
    public static class FluxAmbientBed
    {
        public const float BgmVolume = 0.35f;     // P-27 item 3: BGM below city info
        public const float AmbientVolume = 0.5f;  // P-27 item 3: ambient/weather layer
        public const string NoiseBedClip   = "Assets/Audio/ambience/busy_cyberworld.ogg";
        public const string BgmDawnClip    = "Assets/Audio/music/calm_synthwave_421k.mp3";
        public const string BgmDayClip     = "Assets/Audio/music/synth_wave_0.mp3";
        public const string BgmDuskClip    = "Assets/Audio/music/tt_caves.ogg";
        public const string BgmNightClip   = "Assets/Audio/music/midnight_drive.ogg";
        public const string WeatherRainClip  = "Assets/Audio/weather/rain_loop_2.ogg";
        public const string WeatherStormClip = "Assets/Audio/weather/rain_loop_3.ogg";
        public const string WeatherWindClip  = "Assets/Audio/weather/wind2.wav";
        public const float WindLayerMs = 10.8f;   // Beaufort 6 lower bound
        public static string BgmFor(AmbientTier t)
        {
            switch (t)
            {
                case AmbientTier.Dawn: return BgmDawnClip;
                case AmbientTier.Day:  return BgmDayClip;
                case AmbientTier.Dusk: return BgmDuskClip;
                default:               return BgmNightClip;
            }
        }
        public static string WeatherFor(WeatherMode mode, bool alert, float windMs)
        {
            if (mode == WeatherMode.Rain) return alert ? WeatherStormClip : WeatherRainClip;
            if (mode == WeatherMode.Snow) return null;   // no snow asset: silence (honesty)
            if (windMs &gt;= WindLayerMs) return WeatherWindClip;
            return null;
        }
    }
}
```

```
// BrainCrownRules.cs
// FluxVerse r177 (T-FV-002 S4, CEO order 09-26 00:10 art rectification):
// brain-tower magnolia-bud crown - pure rules table (FacadeRules family
// idiom; person/city data stays out - r10 stale-copy law). Source law =
// Tools/city/landmarks-manifest.json crown node (the r176 sandbox, 95
// assertions green - single geometry source; BrainCrownProof mirrors it).
// Family law: static built-city scenery persisted in the scene like
// signs/offices/facades (NeonSigns law); NOT CityAmbient runtime children
// (r146 red-chain-3 law). order 3 = the building family tier
// (LabsRules.PodOrder law), z 0 tilemap plane; rim 5 and signs 6 ride ABOVE
// by law (pre-registered interplay, r169 A4b report law - the dusk rim
// band over the bud top is intended stacking, never gated here).
// Reshape-only law (r174 note 5a): the crown lands INSIDE the tip zone with
// ZERO added height - canvas top 19.0 == tower top 19.0 &lt; zenith floor
// 19.04 (r131 A10 law). Containment law: the crown rect is CONTAINED in
// NeonRules.Buildings[7] whole [-2,9,3,19] (tower-ornament identity law -
// containment is the gate, disjointness would be a bug). Tint exemption
// (r132 tint_band_note): the crown sits ABOVE the r51 tint ceiling +15 =
// by-design tower-top glow zone; FacadeRules.InTintBand is NOT a crown gate.
// Unlit law (r175 bake gate): bright &gt; 80 only on the two converging sepal
// ridges in Lucy-blue (43 px, 96/160/255 - the r140 spectral anchor); the
// crown carries zero static lit windows (lit rate = P-38(3) heat line).
// Needle law: the side antenna needles dip 0.006u into the canvas zone over
// TRANSPARENT columns only; the mid needle lands on the bud tip where the
// lit ridges converge (NeonRules.NeonTowerAntM/L/R, order 6 above).
// ASCII. No 3D.
using UnityEngine;
namespace FluxVerse
{
    public static class BrainCrownRules
    {
        public const int Count = 1;
        public const int Order = 3;        // building family tier (LabsRules.PodOrder law)
        public const float Z = 0f;         // tilemap plane (facade family z-tie idiom)
        public const float PPU = 24f;      // office-towers importer row (r151, never assumed)
        public const string NamePrefix = "BrainCrown";
        public const string Name = "BrainCrown";
        public const string Path = "Assets/ArtPacks/office-towers/brain-crown.png";
        public const int PxW = 48;         // 2u at PPU24 (natural size law, zero resample)
        public const int PxH = 24;         // 1u
        public const float X0 = -0.5f;     // canvas centered on the tip cell x-center (x0.5)
        public const float Y0 = 18f;       // tip zone floor == shoulder row top plane
        public const float X1 = 1.5f;
        public const float Y1 = 19f;       // == tower top (reshape-only, zero added height)
        public const float PosX = 0.5f;    // rect center (pivot center law)
        public const float PosY = 18.5f;
        // host tower row (containment source, NeonRules.Buildings[7])
        public const int HostBuildingIndex = 7;
        // r131 A10 zenith floor: the crown top must stay strictly under it
        public const float ZenithFloor = 19.04f;
```

```
        // r175 bake census pins (pack edit = fail-loud in BrainCrownProof A1)
        public const int OpaquePx = 602;
        public const int LitPx = 43;       // bright &gt; 80 = the Lucy-blue sepal ridges only
        // grounding pin: bud base rows (image bottom, world y~18.04..18.08)
        // opaque span = world x [-0.25, 1.25] (px 6..41 of 48)
        public const int BaseRowPx0 = 6;
        public const int BaseRowPx1 = 41;
        // bud tip pin: image top row (world y~18.96..19) opaque span =
        // px 21..26 -&gt; world [0.375, 0.625]; the side-needle x-bands
        // (world [0.0875,0.2125] / [0.7875,0.9125] = px 14..17 / 31..33)
        // must sit over TRANSPARENT columns there (zero needle occlusion)
        public const int TipRowPx0 = 21;
        public const int TipRowPx1 = 26;
        public static Vector2 Pos() { return new Vector2(PosX, PosY); }
        public static float WorldW() { return X1 - X0; }
        public static float WorldH() { return Y1 - Y0; }
        public static Rect WorldRect() { return new Rect(X0, Y0, WorldW(), WorldH()); }
    }
}
// CameraRig.cs
// FluxVerse P-17 r14: dual-mode camera pure cores (CEO order 18:40, ledger
// P-2026-09-23-17). Spec (TECH section 9 P-17): ortho camera, PPU 16 tiles;
// L0 panorama size 20 (view 71x40 tiles, whole river-band city, slow drift) &lt;-&gt;
// L1 street size 9 (view 32x18 tiles, building faces / robots / interior
// windows). Switch is a 1.2 s eased interpolation of BOTH size and position -
// hard cuts are forbidden. Split: this file holds the headless-testable cores
// (RigMath easing+clamping, RigTween simulation); the MonoBehaviour adapter
// CityCameraRig lives in CityCameraRig.cs (SEPARATE FILE LAW, r14: a component
// class must match its .cs file name or the saved scene reference dies across
// editor sessions). Pure 2D, no 3D.
using System;
using UnityEngine;
namespace FluxVerse
{
    public enum CamLevel { L0Panorama, L1Street }
    // pure logic: constants + easing + band clamping (headless-testable)
    public static class RigMath
    {
        public const float L0Size = 20f;              // panorama: 71x40 tile view
        public const float L1Size = 9f;               // street: 32x18 tile view
        public const float SwitchSeconds = 1.2f;      // CEO spec: eased, no hard cut
        public const float Aspect = 16f / 9f;        // render target aspect (r13 RT 1920x1080)
        // painted city band (mirror of CitySkeletonBuilder: ground rows -16..14, x -50..50)
        public const float BandYMin = -16f, BandYMax = 14f;
        public const float BandXHalf = 34f;           // keeps the 32-wide L1 view inside painted x
        public const float DriftAmpX = 2.5f, DriftAmpY = 1.0f;     // gentle L0 wander
        public const float DriftPeriodX = 48f, DriftPeriodY = 37f; // non-synced = organic
        public static float EaseInOut(float t)   // smoothstep: zero velocity at both ends
        {
            float k = Mathf.Clamp01(t);
```

```
            return k * k * (3f - 2f * k);
        }
        // L1 focus point -&gt; camera center so the view stays inside the painted band
        public static Vector2 ClampFocus(Vector2 p, float halfH, float halfW)
        {
            float y = Mathf.Clamp(p.y, BandYMin + halfH, BandYMax - halfH);
            float x = Mathf.Clamp(p.x, -BandXHalf, BandXHalf);
            return new Vector2(x, y);
        }
        public static Vector2 DriftOffset(float t)   // L0 panorama slow drift, t in seconds
        {
            return new Vector2(Mathf.Sin(t * 6.2831853f / DriftPeriodX) * DriftAmpX,
                               Mathf.Sin(t * 6.2831853f / DriftPeriodY) * DriftAmpY);
        }
    }
    // pure simulation: camera center + ortho size over time (headless-testable).
    // Target may move while the tween runs (L0 drift is a live target): Step
    // interpolates from the frozen start toward the CURRENT target, so landing
    // on a moving target is seamless - that is how hard cuts stay impossible.
    public class RigTween
    {
        public Vector2 Pos { get; private set; }
        public float Size { get; private set; }
        public Vector2 FromPos, ToPos;   // ToPos/ToSize mutable = moving-target support
        public float FromSize, ToSize;
        float elapsed, duration;
        bool moving;
        public bool Moving { get { return moving; } }
        public void Snap(Vector2 pos, float size)
        {
            Pos = pos; Size = size; moving = false;
        }
        public void MoveTo(Vector2 toPos, float toSize, float seconds)
        {
            FromPos = Pos; FromSize = Size;
            ToPos = toPos; ToSize = toSize;
            elapsed = 0f; duration = Mathf.Max(0.0001f, seconds);
            moving = true;
        }
        public void Step(float dt)
        {
            if (!moving) return;
            elapsed += dt;
            float e = RigMath.EaseInOut(elapsed / duration);
            Pos = Vector2.Lerp(FromPos, ToPos, e);
            Size = Mathf.Lerp(FromSize, ToSize, e);
            if (elapsed &gt;= duration) { Pos = ToPos; Size = ToSize; moving = false; }
        }
    }
    // scene adapter CityCameraRig lives in CityCameraRig.cs (SEPARATE FILE LAW, r14)
```

```
}
// CityAmbient.cs
// FluxVerse r14: CityAmbient - thin MonoBehaviour adapter of the ambient cycle (r13 logic).
// SEPARATE FILE LAW (r14): MonoBehaviour classes must live in a &lt;ClassName&gt;.cs file or the
// serialized scene reference breaks across editor sessions (Tuanjie writes embedded
// class-name stubs for mismatched names which do not re-resolve; GetComponent returned
// null on reload and r13's fallback silently stacked 5 ghost CityAmbient objects).
// Logic cores (AmbientWheel / WeatherRules / WeatherField) stay in AmbientWeather.cs.
// r146 (P-71(3) slice B): the mood-visual face rides this same poll - the
// city mood director (MoodDirector.cs, r144 mirror of BigLife
// mood_director.py) derives the five-state mood from the SAME world inputs
// CityBubbles reads (beijing clock + weather_kind + 24h event density +
// WEATHER_ALERT - an INDEPENDENT same-path read, zero coupling), then
// MoodVisualRules maps it onto two closed-band channels: the 17 modulated
// neon signs (grayscale runtime tint, exempt structure family untouched)
// and the router breath peak (BreathPeakScale). RestoreMoodNeutral() is the
// r124 runtime law face: neutral white + scale 1.0 before any scene save.
// r148 (P-38(1)): dusk roofline rim bands ride the same runtime-only lifecycle -
// 11 top-edge strips (RimLightRules mirror of the r147 manifest) built as
// children in EnsureVisuals, tier-gated in ApplyAmbient (alpha = falloff
// strength at dusk, hard zero otherwise), released with everything else in
// ReleaseVisuals (r146 owned-lifetime law).
// r156 (P-20260925-09 D1): tier flips now EASE. The play path is TransitionAmbient -
// the five-value palette family (skyTop/skyBottom/tint rgb/tintAlpha/camBg) blends over
// AmbientBlend.BlendSeconds with the r14 zero-end-velocity easing, so the four-tier
// wheel cross-fades instead of hard-cutting (the "feel" defect, r153). ApplyAmbient
// stays the INSTANT settle primitive (boot build + all proof call sites) - the two
// entries converge on identical visuals at settle, which AmbientProof asserts as the
// parity/no-pop gate. Play pumps the blend from Update -&gt; StepAmbient; proofs pump
// the SAME Advance core directly (CameraRig law).
// r158 (P-20260925-09 D4): two dawn/dusk horizon glow quads join this runtime-only
// family (LightFxRules.Horizon* mirror of the lightfx manifest v0.2). The glow rides
// the SAME D1 blend - color = the interpolated palette's skyBottom family, alpha
// eased with the same zero-end-velocity curve (never a hard cut, manifest blend_ride
// law); ApplyAmbient settles it instantly like every other instant-family face.
// r181 (T-FV-002 S6b, duskgold-manifest.depth_fog_wash): the depth fog wash quad
// joins the same runtime-only family - one dusk-only mauve veil over the north
// bank (atmospheric perspective, census-triggered), constant SkylineRules.FogFar
// dusk tint, alpha riding the same blend window (variant B: tile-city-only).
// r206 (T-FV-122 S2b, batch-2 order item 1): the tint band splits into THREE
// blocks (TintBandRules): city south / river / city north. The RIVER block is
// EXEMPT at dusk and dawn (the authored blue-purple water reads rose under the
// warm tint in linear color space - r205 forensics) and rides its own alpha
// window through the D1 blend (night carries the wheel alpha; the water
// darkens into the night with the same zero-end-velocity curve). The city
// blocks keep the r13/r51 four-tier wheel law untouched.
// Polls world/world-state.json READ-ONLY every ~10s (perceptor owns all writes).
// Pure 2D: sky/tint/band/fog = SpriteRenderer quads; rain/snow = recycled sprite field.
using System;
using System.IO;
```

```
using System.Text.RegularExpressions;
using UnityEngine;
namespace FluxVerse
{
    // scene component: polls world-state.json read-only and drives all ambient visuals
    public class CityAmbient : MonoBehaviour
    {
        public const float PollIntervalSec = 10f;
        const int RainDrops = 140, SnowDrops = 90;
        [Serializable] class StateFile { public Reality reality; }
        [Serializable] class Reality
        {
            public string city_day_phase, beijing_hhmm, weather_kind, weather_code, weather_wind_ms;
        }
        // P-28 item 2 (r34): parallax silhouette sprites (scene-wired serialized fields;
        // unwired = silent degrade per the probe contract - no skyline, no exceptions)
        public Sprite skylineFar, skylineNear;
        WeatherField field;
        SpriteRenderer sky, tintS, tintN, tintRiver, band;   // r206 S2b: tint = three blocks
        SpriteRenderer skylineFarR, skylineNearR;
        SpriteRenderer[] drops;
        SpriteRenderer[] rims;        // r148: 11 dusk roofline rim bands (runtime-only)
        SpriteRenderer[] horizon;     // r158: 2 dawn/dusk horizon glow quads (runtime-only)
        float horizonA, horizonAFrom, horizonATo;   // r158: horizon alpha blend state (rides the D1 window)
        SpriteRenderer fogWash;       // r181: depth fog wash quad (runtime-only, duskgold-manifest)
        float fogA, fogAFrom, fogATo;               // r181: fog alpha blend state (rides the D1 window)
        float riverA, riverAFrom, riverATo;         // r206 S2b: river block alpha (rides the same window)
        float pollTimer = 999f;      // poll on first Update
        float bandPhase, bandAlpha;
        AmbientTier tier = AmbientTier.Night;
        WeatherMode mode = WeatherMode.None;
        bool alert;
        string curMood;               // r146: applied mood name (null = neutral / not yet polled)
        AmbientBlend blend = new AmbientBlend();                                    // r156 D1
        AmbientPalette curPal = AmbientWheel.PaletteFor(AmbientTier.Night);        // r156: effective palette
        static Texture2D _blendTex;   // r156: runtime sky gradient during a blend
        static Sprite _blendSky;      // r156: its sprite (same 8x64 construction as SkySprite)
        // r146: independent stream read (CityBubbles same-path law - identical
        // regexes, single backward pass, ts-unparseable lines never count)
        static readonly Regex MoodTypeRx = new Regex("\"type\":\"([A-Z_]+)\"");
        static readonly Regex MoodTsRx = new Regex("\"ts_utc\":\"([^\"]+)\"");
        public AmbientTier CurrentTier { get { return tier; } }
        public WeatherMode CurrentMode { get { return mode; } }
        public bool AlertOn { get { return alert; } }
        public float BandAlpha { get { return bandAlpha; } }
        public float WindMs { get; private set; }   // r31: read-only for the ambient bed adapter
        public string CurrentMood { get { return curMood; } }   // r146 proof tap
        public bool BlendActive { get { return blend.Active; } }              // r156 D1 proof tap
        public Sprite CurrentSkySprite { get { return sky != null ? sky.sprite : null; } }   // r156 proof tap
        public float CurrentHorizonAlpha { get { return horizonA; } }          // r158 proof tap
```

```
public float CurrentFogAlpha { get { return fogA; } }                 // r181 proof tap
public float CurrentRiverAlpha { get { return riverA; } }             // r206 S2b proof tap
// r156 proof tap: the cached per-tier gradient sprite (settle/no-pop parity gate)
public static Sprite CachedSkyFor(AmbientTier t) { return SkySprite(t); }
void Awake() { EnsureVisuals(); }
void Update()
{
    pollTimer += Time.deltaTime;
    if (pollTimer &gt;= PollIntervalSec) { pollTimer = 0f; Poll(); }
    StepAmbient(Time.deltaTime);   // r156 D1: play pumps the palette blend
    StepWeather(Time.deltaTime);
    SyncSkyline();
}
// reads the perceptor snapshot (read-only); silent-degrade on any file trouble
public void Poll()
{
    try
    {
        StateFile sf = JsonUtility.FromJson&lt;StateFile&gt;(File.ReadAllText(StatePath()));
        if (sf == null || sf.reality == null) return;
        Reality r = sf.reality;
        int h = ParseHour(r.beijing_hhmm);
        AmbientTier t = h &gt;= 0 ? AmbientWheel.TierForHour(h)
                                 : AmbientWheel.TierFromName(r.city_day_phase);
        TransitionAmbient(t);   // r156 D1: play path eases the tier flip
        ApplyWeather(r.weather_kind, ParseFloat(r.weather_wind_ms), (int)ParseFloat(r.weather_code));
        ApplyMoodVisual(DeriveMoodFromWorld());   // r146: same poll, mood-visual face
    }
    catch (Exception) { /* keep current visuals: probe contract silent degrade */ }
}
// r146 (P-71(3) slice B): derive the city mood from the world inputs -
// the same files CityBubbles reads, read INDEPENDENTLY (zero coupling,
// deterministic same-input-same-output). Null = mood absent (calendar
// null / file trouble) = the steady row downstream (degrade law).
public string DeriveMoodFromWorld()
{
    try
    {
        StateFile sf = JsonUtility.FromJson&lt;StateFile&gt;(File.ReadAllText(StatePath()));
        if (sf == null || sf.reality == null) return null;
        string hhmm = sf.reality.beijing_hhmm;
        int h, m;
        if (string.IsNullOrEmpty(hhmm) || !ParseHourMin(hhmm, out h, out m)) return null;
        string repo = Path.GetDirectoryName(Path.GetDirectoryName(Application.dataPath));
        int density; bool alertHit;
        ReadStreamFacts(Path.Combine(repo, "world", "world-events.jsonl"), out density, out alertHit);
        DateTime bj = DateTime.Now.Date + new TimeSpan(h, m, 0);
        MoodState st = MoodDirector.DeriveState(bj, density, MoodDirector.DenseDefault,
            alertHit, sf.reality.weather_kind, MoodDirector.LoadCalendar());
        return st != null ? st.Mood : null;
```

```
    }
    catch (Exception) { return null; }
}
// r146: apply the mood row to both channels (the proof calls this
// directly with a pinned mood; the live poll feeds DeriveMoodFromWorld).
// Neon channel = the 17 modulated table signs (fresh Find per apply =
// flip-safe against any rebuild; exempt structure family never touched);
// breath channel = the persistent CityEventRouter's core scale. Missing
// GOs degrade silently here - the PROOF fails loud instead.
public void ApplyMoodVisual(string moodName)
{
    float neon = MoodVisualRules.NeonScaleFor(moodName);
    float breath = MoodVisualRules.BreathScaleFor(moodName);
    curMood = moodName;
    Color moodTint = MoodVisualRules.NeonTint(neon);
    for (int i = 0; i &lt; NeonRules.Count; i++)
    {
        if (MoodVisualRules.IsExemptSign(i)) continue;
        GameObject go = GameObject.Find(NeonRules.Name(i));
        if (go == null) continue;
        SpriteRenderer sr = go.GetComponent&lt;SpriteRenderer&gt;();
        if (sr == null) continue;
        sr.color = moodTint;
    }
    CityEventRouter cer = FindEventRouter();
    if (cer != null) cer.Core.BreathPeakScale = breath;
}
// r124 runtime law: restore neutral white signs + baseline breath scale
// before any scene save - the saved scene never learns runtime mood.
public void RestoreMoodNeutral()
{
    for (int i = 0; i &lt; NeonRules.Count; i++)
    {
        if (MoodVisualRules.IsExemptSign(i)) continue;
        GameObject go = GameObject.Find(NeonRules.Name(i));
        if (go == null) continue;
        SpriteRenderer sr = go.GetComponent&lt;SpriteRenderer&gt;();
        if (sr != null) sr.color = Color.white;
    }
    CityEventRouter cer = FindEventRouter();
    if (cer != null) cer.Core.BreathPeakScale = 1f;
    curMood = null;
}
static CityEventRouter FindEventRouter()
{
    GameObject go = GameObject.Find("CityEventRouter");
    return go != null ? go.GetComponent&lt;CityEventRouter&gt;() : null;
}
// r146 (r23 owned-lifetime law face): destroy every runtime visual child.
// All ambient visuals (sky/skyline/tint/band/drops/rims/horizon) are runtime-only
```

```
// children by the r13/r25 law - nothing is ever serialized - so ANY child
// on disk is stale contamination (the r146 red-chain: a restore-save once
// persisted EnsureVisuals children into CityScene). Proofs call this
// before any scene save; EnsureVisuals() rebuilds everything after it.
public void ReleaseVisuals()
{
    for (int i = transform.childCount - 1; i &gt;= 0; i--)
    {
        Transform c = transform.GetChild(i);
        if (Application.isPlaying) Destroy(c.gameObject);
        else DestroyImmediate(c.gameObject);
    }
    sky = null; tintS = null; tintN = null; tintRiver = null; band = null;   // r206 S2b: three blocks die with
    skylineFarR = null; skylineNearR = null;
    drops = null;
    rims = null;               // r148: rim bands die with the family
    horizon = null;            // r158: horizon quads die with the family
    fogWash = null;            // r181: the fog wash dies with the family
    field = null;
}
// r146: 24h event density + WEATHER_ALERT hit from the live stream
// (CityBubbles.ReadStream mirror - backward pass, rotation-safe, the
// perceptor owns writes). Mood_director.py read_world law: a line with
// an unparseable ts never counts - data absent cannot be verified.
static void ReadStreamFacts(string streamPath, out int density, out bool alertHit)
{
    density = 0; alertHit = false;
    if (!File.Exists(streamPath)) return;
    string[] lines;
    try { lines = File.ReadAllLines(streamPath); }
    catch (Exception) { return; }
    DateTime cut = DateTime.UtcNow.AddHours(-24.0);
    for (int i = lines.Length - 1; i &gt;= 0; i--)
    {
        string ln = lines[i];
        if (string.IsNullOrEmpty(ln)) continue;
        if (TsInWindow(ln, cut))
        {
            density++;
            if (!alertHit &amp;&amp; MoodTypeRx.Match(ln).Groups[1].Value == "WEATHER_ALERT") alertHit = true;
        }
    }
}
static bool TsInWindow(string line, DateTime cut)
{
    Match m = MoodTsRx.Match(line);
    if (!m.Success) return false;
    DateTime t;
    if (!DateTime.TryParseExact(m.Groups[1].Value, "yyyy-MM-dd'T'HH:mm:ss'Z'",
            System.Globalization.CultureInfo.InvariantCulture,
```

```
            System.Globalization.DateTimeStyles.AssumeUniversal, out t))
        return false;
    return t &gt;= cut;
}
static bool ParseHourMin(string hhmm, out int h, out int m)
{
    h = 0; m = 0;
    int sep = hhmm.IndexOf(':');
    if (sep &lt;= 0) return false;
    return int.TryParse(hhmm.Substring(0, sep), out h) &amp;&amp; int.TryParse(hhmm.Substring(sep + 1), out m);
}
static string StatePath()
{
    string projectRoot = Path.GetDirectoryName(Application.dataPath);   // .../City
    string repoRoot = Path.GetDirectoryName(projectRoot);               // .../FluxVerse
    return Path.Combine(repoRoot, "world", "world-state.json");
}
static int ParseHour(string hhmm)
{
    if (string.IsNullOrEmpty(hhmm)) return -1;
    int sep = hhmm.IndexOf(':');
    if (sep &lt;= 0) return -1;
    int h; return int.TryParse(hhmm.Substring(0, sep), out h) ? h : -1;
}
static float ParseFloat(string s)
{
    float v; return float.TryParse(s, out v) ? v : 0f;
}
// r156 D1: INSTANT settle primitive - boot build, proofs (53 call sites), and the
// tier-gated families. The play path is TransitionAmbient (eased tier flips).
public void ApplyAmbient(AmbientTier t)
{
    if (sky == null) EnsureVisuals();
    tier = t;
    horizonAFrom = horizonATo = LightFxRules.HorizonAlphaFor(t);   // r158: instant path settles at once
    fogAFrom = fogATo = LightFxRules.FogWashAlphaFor(t);            // r181: fog settles with the same law
    riverAFrom = riverATo = TintBandRules.RiverAlphaFor(t);        // r206 S2b: river settles at once (night=whe
    ApplyInstantFamilies(t);
    SettleAt(t);
}
// r156 D1: PLAY-path tier entry - ease the five-value palette family over
// BlendSeconds instead of the hard cut. Skyline fog + rims stay instant per
// their own family laws (r34 fog gates / r148 manifest tier_gate); the
// r158 horizon glow rides the blend window (manifest blend_ride law).
public void TransitionAmbient(AmbientTier t)
{
    if (sky == null) EnsureVisuals();
    tier = t;
    horizonAFrom = horizonA;                        // r158: ride from the CURRENT applied value
    horizonATo = LightFxRules.HorizonAlphaFor(t);   // eases with the D1 blend window
```

```
    fogAFrom = fogA;                                // r181: the fog rides the same window
    fogATo = LightFxRules.FogWashAlphaFor(t);
    riverAFrom = riverA;                            // r206 S2b: the river rides the same window
    riverATo = TintBandRules.RiverAlphaFor(t);
    AmbientPalette target = AmbientWheel.PaletteFor(t);
    ApplyInstantFamilies(t);
    if (PaletteNear(curPal, target)) { SettleAt(t); return; }   // same tier = no-op settle
    blend.Begin(curPal, target, AmbientBlend.BlendSeconds);
    sky.sprite = BlendSky();   // runtime gradient takes over until settle
}
void SettleAt(AmbientTier t)
{
    blend.End();
    curPal = AmbientWheel.PaletteFor(t);
    sky.sprite = SkySprite(t);
    ApplyPalette(curPal);
}
// five-value family application (the interpolated face)
void ApplyPalette(AmbientPalette p)
{
    curPal = p;
    // r206 S2b: the two CITY blocks keep the four-tier wheel law; the
    // RIVER block rides its own alpha window (dusk/dawn exempt = the
    // authored blue-purple reads; night carries the wheel alpha - the
    // water darkens into the night on the same eased curve).
    Color cityTint = new Color(p.tint.r, p.tint.g, p.tint.b, p.tintAlpha);
    if (tintS != null) tintS.color = cityTint;
    if (tintN != null) tintN.color = cityTint;
    if (tintRiver != null)
    {
        float kr = blend.Active ? RigMath.EaseInOut(blend.Progress01) : 1f;
        riverA = Mathf.Lerp(riverAFrom, riverATo, kr);
        tintRiver.color = new Color(p.tint.r, p.tint.g, p.tint.b, riverA);
    }
    Camera cam = Cam();
    if (cam != null) cam.backgroundColor = p.camBg;
    if (blend.Active &amp;&amp; _blendTex != null)
    {
        // repaint the runtime sky gradient - the SAME 8x64 construction as
        // SkySprite, so the settle swap back to the cached sprite is
        // content-identical (zero pop by construction)
        const int W = 8, H = 64;
        for (int y = 0; y &lt; H; y++)
        {
            Color c = Color.Lerp(p.skyBottom, p.skyTop, y / (float)(H - 1));   // y=0 bottom row
            for (int x = 0; x &lt; W; x++) _blendTex.SetPixel(x, y, c);
        }
        _blendTex.Apply();
    }
    if (horizon != null)
```

```
        {
            // r158: the horizon glow rides the SAME blend - color = the
            // interpolated palette's skyBottom family, alpha eased with the
            // same zero-end-velocity curve (never a hard cut, manifest law)
            float k = blend.Active ? RigMath.EaseInOut(blend.Progress01) : 1f;
            horizonA = Mathf.Lerp(horizonAFrom, horizonATo, k);
            Color hc = new Color(p.skyBottom.r, p.skyBottom.g, p.skyBottom.b, horizonA);
            for (int i = 0; i &lt; horizon.Length; i++)
                if (horizon[i] != null) horizon[i].color = hc;
        }
        if (fogWash != null)
        {
            // r181: the depth fog rides the SAME blend window - constant
            // mauve tint (the SkylineRules.FogFar dusk family, single
            // source), alpha eased with the identical curve
            float kf = blend.Active ? RigMath.EaseInOut(blend.Progress01) : 1f;
            fogA = Mathf.Lerp(fogAFrom, fogATo, kf);
            Color fc = LightFxRules.FogWashColor();
            fogWash.color = new Color(fc.r, fc.g, fc.b, fogA);
        }
    }
    // r156 D1: advance the palette blend. Play pumps this from Update; proofs pump
    // the same core directly - one Advance law for both (CameraRig pattern).
    public void StepAmbient(float dt)
    {
        if (!blend.Active) return;
        ApplyPalette(blend.Advance(dt));
        if (!blend.Active) sky.sprite = SkySprite(tier);   // settle: back to the cached tier sprite
    }
    // tier-gated families that stay INSTANT on every flip (their own family laws:
    // r34 fog gates; r148 rim manifest tier_gate - per-art-row dither alphas live
    // in the texture, this alpha carries the strength, honey orange = ambient
    // channel riding the dusk tint exactly like the signs)
    void ApplyInstantFamilies(AmbientTier t)
    {
        if (skylineFarR != null) skylineFarR.color = SkylineRules.FogFar(t);
        if (skylineNearR != null) skylineNearR.color = SkylineRules.FogNear(t);
        if (rims != null)
        {
            for (int i = 0; i &lt; rims.Length; i++)
            {
                Color c = RimLightRules.BaseColorFor(i);
                rims[i].color = new Color(c.r, c.g, c.b, RimLightRules.AlphaFor(i, t));
            }
        }
    }
    static bool PaletteNear(AmbientPalette a, AmbientPalette b)
    {
        const float e = 0.0005f;
        return Mathf.Abs(a.skyTop.r - b.skyTop.r) &lt; e &amp;&amp; Mathf.Abs(a.skyTop.g - b.skyTop.g) &lt; e &amp;&
```

```
        &amp;&amp; Mathf.Abs(a.skyBottom.r - b.skyBottom.r) &lt; e &amp;&amp; Mathf.Abs(a.skyBottom.g - b.skyBot
        &amp;&amp; Mathf.Abs(a.tint.r - b.tint.r) &lt; e &amp;&amp; Mathf.Abs(a.tint.g - b.tint.g) &lt; e &amp;&
        &amp;&amp; Mathf.Abs(a.tintAlpha - b.tintAlpha) &lt; e
        &amp;&amp; Mathf.Abs(a.camBg.r - b.camBg.r) &lt; e &amp;&amp; Mathf.Abs(a.camBg.g - b.camBg.g) &lt; e &a
}
// r156 D1: the blend-window sky gradient scaffold (identical geometry to the
// cached tier sprites: 8x64 @ PPU16 -&gt; the quad scale never changes on swap)
static Sprite BlendSky()
{
    if (_blendSky != null) return _blendSky;
    _blendTex = new Texture2D(8, 64, TextureFormat.RGBA32, false);
    _blendTex.wrapMode = TextureWrapMode.Clamp;
    _blendTex.filterMode = FilterMode.Bilinear;
    _blendSky = Sprite.Create(_blendTex, new Rect(0, 0, 8, 64), new Vector2(0.5f, 0.5f), 16f);
    return _blendSky;
}
// P-28 (r34): silhouettes track 0.9 x camera (subtle parallax). Y stays glued
// to the world horizon (a distant skyline does not sink when the camera drops).
public void SyncSkyline()
{
    if (skylineFarR == null &amp;&amp; skylineNearR == null) return;
    Camera cam = Cam();
    float fx = cam != null ? SkylineRules.FollowFactor * cam.transform.position.x : 0f;
    if (skylineFarR != null)
    {
        Vector3 p = skylineFarR.transform.position;
        skylineFarR.transform.position = new Vector3(fx, p.y, p.z);
    }
    if (skylineNearR != null)
    {
        Vector3 p = skylineNearR.transform.position;
        skylineNearR.transform.position = new Vector3(fx, p.y, p.z);
    }
}
public void ApplyWeather(string kind, float windMs, int wmoCode)
{
    if (sky == null) EnsureVisuals();
    mode = WeatherRules.ModeForKind(kind);
    alert = WeatherRules.IsAlert(windMs, wmoCode);
    WindMs = windMs;
    field.Configure(mode, windMs);
    SyncDrops();
}
public void StepWeather(float dt)
{
    if (sky == null) EnsureVisuals();
    if (mode != WeatherMode.None)
    {
        field.Step(dt);
        SyncDrops();
```

```
        }
        bandPhase += dt * 2.6f;
        float target = alert ? 0.10f + 0.13f * (0.5f + 0.5f * Mathf.Sin(bandPhase)) : 0f;
        bandAlpha = Mathf.Lerp(bandAlpha, target, Mathf.Clamp01(dt * 4f));
        band.color = new Color(1f, 0.12f, 0.08f, bandAlpha);
    }
    // builds every visual lazily (nothing persisted in the scene asset; runtime children only)
    public void EnsureVisuals()
    {
        if (sky != null) return;
        // sky backdrop: behind every tilemap (builder sorting orders 0..4). Quad height = 40 =
        // exactly the ortho-20 vertical view, so the visible band shows the FULL gradient
        // (zenith color truly reached at the top edge; camBg = skyTop covers any later drift).
        sky = MakeQuad("AmbientSky", -10, SkySprite(tier), 92f, 40f, 0f);
        sky.transform.SetParent(transform, false);
        // P-28 item 2 (r34): parallax silhouettes behind every tilemap, in front of
        // the sky (orders -9/-8). Runtime-constructed like every ambient visual;
        // unwired sprites stay silent. Integer x2 scale = point-filter law (r29 gate).
        if (skylineFar != null)
        {
            skylineFarR = SkylineQuad("SkylineFar", SkylineRules.FarOrder, skylineFar,
                SkylineRules.FarTopRow, SkylineRules.FarContentTopY, SkylineRules.FogFar(tier));
            skylineFarR.transform.SetParent(transform, false);
        }
        if (skylineNear != null)
        {
            skylineNearR = SkylineQuad("SkylineNear", SkylineRules.NearOrder, skylineNear,
                SkylineRules.NearTopRow, SkylineRules.NearContentTopY, SkylineRules.FogNear(tier));
            skylineNearR.transform.SetParent(transform, false);
        }
        // ambient tint over the CITY band only - the PAINTED extent (y -16..+15): the
        // builder paves to world +15 (tile cells to 14), so a 30u quad topping at +14
        // left a 1u full-width untinted bright strip at the far shore on every tinted
        // tier (r22 finding; fixed by the r51 pilot, TECH sec.9 debt line). Atmosphere
        // between buildings, sky strips stay pure gradient. Above tilemaps 0..4, below
        // band 9 / pulses 10.
        // r206 S2b (batch-2 order item 1): the single quad splits into THREE blocks
        // (TintBandRules) - city south (-16..-3) / river (the water world rect
        // exactly) / city north (+3..+15). The river block is EXEMPT at dusk and
        // dawn (the warm tint over the dark blue-purple water reads rose in
        // linear color space - r205 forensics) and carries the wheel alpha at
        // night only; the city blocks keep the r13/r51 law (92u family width,
        // bottom -16, painted top +15). Seams flush by constant.
        tintS = MakeQuad(TintBandRules.NameS, TintBandRules.Order, WhiteSprite(),
            TintBandRules.CityW, TintBandRules.SyH, TintBandRules.SyCy);
        tintS.transform.SetParent(transform, false);
        tintRiver = MakeQuad(TintBandRules.NameRiver, TintBandRules.Order, WhiteSprite(),
            TintBandRules.RiverW, TintBandRules.RiverH, TintBandRules.RiverCy);
        tintRiver.transform.position = new Vector3(TintBandRules.RiverCx, TintBandRules.RiverCy, 0f);
        tintRiver.transform.SetParent(transform, false);
```

```
        tintN = MakeQuad(TintBandRules.NameN, TintBandRules.Order, WhiteSprite(),
            TintBandRules.CityW, TintBandRules.NyH, TintBandRules.NyCy);
        tintN.transform.SetParent(transform, false);
        // city-wide alert band (gale / severe WMO): functional red, river level
        band = MakeQuad("AmbientAlertBand", 9, WhiteSprite(), 92f, 4f, 0f);
        band.transform.SetParent(transform, false);
        band.color = new Color(1f, 0.12f, 0.08f, 0f);
        // weather drop field: front of everything (above event pulses for depth feel)
        drops = new SpriteRenderer[RainDrops];
        for (int i = 0; i &lt; drops.Length; i++)
        {
            GameObject go = new GameObject("AmbientDrop" + i);
            go.transform.SetParent(transform, false);
            SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();
            sr.sortingOrder = 12;
            drops[i] = sr;
            go.SetActive(false);
        }
        field = new WeatherField(RainDrops, 20260923);
        // r148 (P-38(1)): 11 dusk roofline rim bands - procedural per-segment
        // texture (artW x 2 art px white, hard top row + 1-step dither decay,
        // pixel-art law), natural-size build at PPU16 (scale 1, the 1/16u art
        // grid maps 1:1 - zero resampling). Order 5 = free slot above the
        // tilemaps, below signs/tint/band/pulses (CEO pulse stays above, law).
        rims = new SpriteRenderer[RimLightRules.SegmentCount];
        for (int i = 0; i &lt; rims.Length; i++) rims[i] = MakeRimBand(i);
        // r158 (P-20260925-09 D4): two dawn/dusk horizon glow quads - runtime
        // gradient sprite (white, vertical alpha ramp strongest at the horizon
        // line) tinted per tier with the palette skyBottom family; order -9
        // with the DEEPEST z of that family = the sunset glow sits behind
        // the distant silhouettes (physics law r158, manifest v0.2).
        horizon = new SpriteRenderer[LightFxRules.HorizonCount];
        for (int i = 0; i &lt; horizon.Length; i++) horizon[i] = MakeHorizonQuad(i);
        horizonA = horizonAFrom = horizonATo = LightFxRules.HorizonAlphaFor(tier);
        // r181 (duskgold-manifest.depth_fog_wash): one dusk-only atmospheric
        // veil over the north bank - the census-triggered depth fix. Same
        // runtime-only family as the horizon quads (released together).
        fogWash = MakeFogQuad();
        fogA = fogAFrom = fogATo = LightFxRules.FogWashAlphaFor(tier);
        ApplyAmbient(tier);
        SyncDrops();
    }
    // r148: one roofline rim band - a runtime-only child of CityAmbient
    SpriteRenderer MakeRimBand(int i)
    {
        int w = RimLightRules.ArtPxWidth(i);
        Texture2D tex = new Texture2D(w, RimLightRules.DitherRows, TextureFormat.RGBA32, false);
        tex.filterMode = FilterMode.Point;
        for (int y = 0; y &lt; RimLightRules.DitherRows; y++)
        {
```

```
            // texture y=0 is the BOTTOM row; rowFromTop 0 = hard top row
            float a = RimLightRules.DitherRowAlpha(RimLightRules.DitherRows - 1 - y);
            for (int x = 0; x &lt; w; x++) tex.SetPixel(x, y, new Color(1f, 1f, 1f, a));
        }
        tex.Apply();
        Sprite s = Sprite.Create(tex, new Rect(0, 0, w, RimLightRules.DitherRows),
            new Vector2(0.5f, 0.5f), 16f);
        GameObject go = new GameObject("RimBand" + i);
        SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();
        sr.sprite = s;
        sr.sortingOrder = RimLightRules.SortOrder;
        go.transform.position = new Vector3(
            (RimLightRules.X0(i) + RimLightRules.X1(i)) * 0.5f,
            (RimLightRules.Y0(i) + RimLightRules.Y1(i)) * 0.5f, 0f);
        go.transform.SetParent(transform, false);
        return sr;
    }
    // r158: one horizon glow quad - a runtime-only child of CityAmbient.
    // The sprite is a 4x34 vertical alpha ramp (quadratic, strongest at the
    // bottom = the horizon line); scale is RELATIVE per the r13 MakeQuad law.
    SpriteRenderer MakeHorizonQuad(int i)
    {
        float x0 = LightFxRules.HorizonX0(i), x1 = LightFxRules.HorizonX1(i);
        float y0 = LightFxRules.HorizonY0, y1 = LightFxRules.HorizonY1;
        Sprite s = HorizonSprite();
        GameObject go = new GameObject(LightFxRules.HorizonName(i));
        SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();
        sr.sprite = s;
        sr.sortingOrder = LightFxRules.HorizonOrder;
        Vector3 n = s.bounds.size;   // natural world size: scale must be RELATIVE
        go.transform.localScale = new Vector3((x1 - x0) / n.x, (y1 - y0) / n.y, 1f);
        go.transform.position = new Vector3((x0 + x1) * 0.5f, (y0 + y1) * 0.5f,
            LightFxRules.HorizonZ);
        go.transform.SetParent(transform, false);
        return sr;
    }
    static Sprite _horizon;
    static Sprite HorizonSprite()   // 4x34: white, alpha ramp strongest at the bottom
    {
        if (_horizon != null) return _horizon;
        const int W = 4, H = 34;   // 2.125u tall @ ppu16 = the 1/16u art grid band
        Texture2D tex = new Texture2D(W, H, TextureFormat.RGBA32, false);
        tex.filterMode = FilterMode.Point;
        for (int y = 0; y &lt; H; y++)
        {
            float t = y / (float)(H - 1);          // texture y=0 = bottom row
            float a = (1f - t) * (1f - t);         // quadratic: strongest at the horizon line
            for (int x = 0; x &lt; W; x++) tex.SetPixel(x, y, new Color(1f, 1f, 1f, a));
        }
        tex.Apply();
```

```
    _horizon = Sprite.Create(tex, new Rect(0, 0, W, H), new Vector2(0.5f, 0.5f), 16f);
    return _horizon;
}
// r181: the depth fog wash quad - a runtime-only child of CityAmbient.
// Flat white sprite (the shared WhiteSprite, MakeQuad law) with a
// RELATIVE scale to the world band; the mauve tint + tier alpha live
// on sr.color (set by ApplyPalette through the blend ride).
SpriteRenderer MakeFogQuad()
{
    Sprite s = WhiteSprite();
    GameObject go = new GameObject(LightFxRules.FogWashName);
    SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();
    sr.sprite = s;
    sr.sortingOrder = LightFxRules.FogWashOrder;
    Vector3 n = s.bounds.size;   // natural world size: scale must be RELATIVE
    go.transform.localScale = new Vector3(
        (LightFxRules.FogWashX1 - LightFxRules.FogWashX0) / n.x,
        (LightFxRules.FogWashY1 - LightFxRules.FogWashY0) / n.y, 1f);
    go.transform.position = new Vector3(
        (LightFxRules.FogWashX0 + LightFxRules.FogWashX1) * 0.5f,
        (LightFxRules.FogWashY0 + LightFxRules.FogWashY1) * 0.5f,
        LightFxRules.FogWashZ);
    go.transform.SetParent(transform, false);
    return sr;
}
void SyncDrops()
{
    int active = mode == WeatherMode.Rain ? drops.Length : (mode == WeatherMode.Snow ? SnowDrops : 0);
    for (int i = 0; i &lt; drops.Length; i++)
    {
        bool on = i &lt; active;
        if (drops[i].gameObject.activeSelf != on) drops[i].gameObject.SetActive(on);
        if (!on) continue;
        if (mode == WeatherMode.Rain)
        {
            drops[i].sprite = StreakSprite();
            drops[i].color = new Color(0.70f, 0.85f, 1f, 0.55f);   // ambient rain, cyan-white
            drops[i].transform.position = field.Pos(i);
            float lean = -Mathf.Atan2(field.WindX, 45f) * Mathf.Rad2Deg;
            drops[i].transform.rotation = Quaternion.Euler(0f, 0f, lean);
            drops[i].transform.localScale = new Vector3(1.6f, 1.6f, 1f);
        }
        else
        {
            drops[i].sprite = DotSprite();
            drops[i].color = new Color(0.95f, 0.97f, 1f, 0.85f);   // snow
            drops[i].transform.position = field.Pos(i);
            drops[i].transform.rotation = Quaternion.identity;
            drops[i].transform.localScale = new Vector3(1.6f, 1.6f, 1f);
        }
```

```
    }
}
static Camera Cam()
{
    GameObject c = GameObject.Find("CityCamera");
    return c == null ? null : c.GetComponent&lt;Camera&gt;();
}
static SpriteRenderer MakeQuad(string name, int order, Sprite sprite, float sx, float sy, float yOff)
{
    GameObject go = new GameObject(name);
    go.transform.position = new Vector3(0f, yOff, 0f);
    SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();
    sr.sprite = sprite;
    sr.sortingOrder = order;
    Vector3 n = sprite.bounds.size;   // natural world size: scale must be RELATIVE
    go.transform.localScale = new Vector3(sx / n.x, sy / n.y, 1f);
    return sr;
}
// P-28 (r34): skyline quad - integer x2 scale (point law), center solved so the
// measured first content row lands exactly at contentTopY (geometry in SkylineRules)
static SpriteRenderer SkylineQuad(string name, int order, Sprite sprite, int topRow, float contentTopY, Color fo
{
    GameObject go = new GameObject(name);
    SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();
    sr.sprite = sprite;
    sr.sortingOrder = order;
    sr.color = fog;
    go.transform.localScale = new Vector3(SkylineRules.Scale, SkylineRules.Scale, 1f);
    go.transform.position = new Vector3(0f, SkylineRules.QuadCenterY(topRow, contentTopY), 0f);
    return sr;
}
static Sprite _white;
static Sprite WhiteSprite()
{
    if (_white != null) return _white;
    const int S = 4;
    Texture2D tex = new Texture2D(S, S, TextureFormat.RGBA32, false);
    tex.filterMode = FilterMode.Point;
    for (int y = 0; y &lt; S; y++)
        for (int x = 0; x &lt; S; x++) tex.SetPixel(x, y, new Color(1f, 1f, 1f, 1f));
    tex.Apply();
    _white = Sprite.Create(tex, new Rect(0, 0, S, S), new Vector2(0.5f, 0.5f), 16f);
    return _white;
}
static Sprite _streak;
static Sprite StreakSprite()   // 3x14 vertical rain streak, tapered ends
{
    if (_streak != null) return _streak;
    const int W = 3, H = 14;
    Texture2D tex = new Texture2D(W, H, TextureFormat.RGBA32, false);
```

```
            tex.filterMode = FilterMode.Point;
            for (int y = 0; y &lt; H; y++)
                for (int x = 0; x &lt; W; x++)
                {
                    float ay = Mathf.Sin((y + 0.5f) / H * Mathf.PI);              // taper ends
                    float ax = 1f - Mathf.Abs((x + 0.5f) - W * 0.5f) / (W * 0.5f); // bright center
                    tex.SetPixel(x, y, new Color(1f, 1f, 1f, ay * ax));
                }
            tex.Apply();
            _streak = Sprite.Create(tex, new Rect(0, 0, W, H), new Vector2(0.5f, 0.5f), 16f);
            return _streak;
        }
        static Sprite _dot;
        static Sprite DotSprite()   // 5x5 soft snow dot
        {
            if (_dot != null) return _dot;
            const int S = 5;
            Texture2D tex = new Texture2D(S, S, TextureFormat.RGBA32, false);
            tex.filterMode = FilterMode.Point;
            for (int y = 0; y &lt; S; y++)
                for (int x = 0; x &lt; S; x++)
                {
                    float dx = (x + 0.5f) / S * 2f - 1f, dy = (y + 0.5f) / S * 2f - 1f;
                    float a = Mathf.Clamp01(1.15f - Mathf.Sqrt(dx * dx + dy * dy));
                    tex.SetPixel(x, y, new Color(1f, 1f, 1f, a));
                }
            tex.Apply();
            _dot = Sprite.Create(tex, new Rect(0, 0, S, S), new Vector2(0.5f, 0.5f), 16f);
            return _dot;
        }
        static readonly Sprite[] _skyCache = new Sprite[4];
        static Sprite SkySprite(AmbientTier t)   // vertical gradient, one per tier (cached)
        {
            int idx = (int)t;
            if (_skyCache[idx] != null) return _skyCache[idx];
            AmbientPalette p = AmbientWheel.PaletteFor(t);
            const int W = 8, H = 64;
            Texture2D tex = new Texture2D(W, H, TextureFormat.RGBA32, false);
            tex.wrapMode = TextureWrapMode.Clamp;
            tex.filterMode = FilterMode.Bilinear;
            for (int y = 0; y &lt; H; y++)
            {
                Color c = Color.Lerp(p.skyBottom, p.skyTop, y / (float)(H - 1));   // y=0 bottom row
                for (int x = 0; x &lt; W; x++) tex.SetPixel(x, y, c);
            }
            tex.Apply();
            _skyCache[idx] = Sprite.Create(tex, new Rect(0, 0, W, H), new Vector2(0.5f, 0.5f), 16f);
            return _skyCache[idx];
        }
    }
```

```
}
// CityAmbientAudio.cs
// FluxVerse r31 (P-27 item 2): CityAmbientAudio - ambient bed adapter (four-tier BGM
// by city_day_phase + weather layer + always-on city noise). SEPARATE FILE LAW (r14):
// a component class must live in &lt;ClassName&gt;.cs or the saved scene reference dies
// across editor sessions (Tuanjie writes embedded class-name stubs that never
// re-resolve). Pure core = FluxAmbientBed (AudioRouter.cs, headless-testable).
// State source = the CityAmbient component on the same GameObject (tier/mode/alert/
// wind read-only getters; same 10s world-state poll, no second file read).
// Play-mode-only law (UiKit runtime law): the loop-bed GameObject is constructed at
// runtime and NEVER saved into the scene; in edit/batch mode ApplyBed updates the
// selection state only and stays silent (bed proofs run in edit mode on that state).
// Unwired clip field = that layer stays silent (honesty law, never a fallback
// jingle). Audible verification = play mode (batch has no audio device, r25 honesty
// boundary); listening calibration = P-27 item 6 v2. Pure 2D. All comments ASCII.
using UnityEngine;
namespace FluxVerse
{
    public class CityAmbientAudio : MonoBehaviour
    {
        public AudioClip noiseBed;                              // ambience/busy_cyberworld
        public AudioClip bgmDawn, bgmDay, bgmDusk, bgmNight;    // music x4 by day phase
        public AudioClip weatherRain;                           // rain_loop_2 (moderate)
        public AudioClip weatherStorm;                          // rain_loop_3 (gale/severe)
        public AudioClip weatherWind;                            // wind2 (dry Beaufort 6+)
        public string CurrentBgm { get; private set; }        // resolved path, "" = silent
        public string CurrentWeather { get { return currentWeather; } }
        public bool BedActive { get { return bgmSrc != null; } }
        string currentWeather = "";
        AudioSource bgmSrc, noiseSrc, weatherSrc;
        string appliedBgm, appliedWeather;   // null sentinels: never equal a real selection
        CityAmbient ambient;
        public AudioClip ResolveClip(string assetPath)
        {
            if (assetPath == FluxAmbientBed.NoiseBedClip) return noiseBed;
            if (assetPath == FluxAmbientBed.BgmDawnClip) return bgmDawn;
            if (assetPath == FluxAmbientBed.BgmDayClip) return bgmDay;
            if (assetPath == FluxAmbientBed.BgmDuskClip) return bgmDusk;
            if (assetPath == FluxAmbientBed.BgmNightClip) return bgmNight;
            if (assetPath == FluxAmbientBed.WeatherRainClip) return weatherRain;
            if (assetPath == FluxAmbientBed.WeatherStormClip) return weatherStorm;
            if (assetPath == FluxAmbientBed.WeatherWindClip) return weatherWind;
            return null;
        }
        // single selection path for play mode AND edit-mode proofs (proof calls this
        // directly with the same signature; Update only feeds it from CityAmbient)
        public void ApplyBed(AmbientTier tier, WeatherMode mode, bool alert, float windMs)
        {
            CurrentBgm = ResolveOrNull(FluxAmbientBed.BgmFor(tier));
            currentWeather = ResolveOrNull(FluxAmbientBed.WeatherFor(mode, alert, windMs));
```

```
    if (!Application.isPlaying) return;   // edit/batch: selection state only (silent law)
    EnsureBed();
    if (CurrentBgm != appliedBgm)
    {
        SetLoop(bgmSrc, CurrentBgm, FluxAmbientBed.BgmVolume);
        appliedBgm = CurrentBgm;
    }
    if (CurrentWeather != appliedWeather)
    {
        SetLoop(weatherSrc, CurrentWeather, FluxAmbientBed.AmbientVolume);
        appliedWeather = CurrentWeather;
    }
}
void Update()
{
    if (!Application.isPlaying) return;
    if (ambient == null) ambient = FindObjectOfType&lt;CityAmbient&gt;();
    if (ambient == null) return;   // no state source yet: stay silent (honesty)
    ApplyBed(ambient.CurrentTier, ambient.CurrentMode, ambient.AlertOn, ambient.WindMs);
}
string ResolveOrNull(string assetPath)
{
    if (string.IsNullOrEmpty(assetPath)) return "";
    return ResolveClip(assetPath) != null ? assetPath : "";   // unwired = silent
}
void EnsureBed()
{
    if (bgmSrc != null) return;
    GameObject go = new GameObject("CityAmbientAudio");   // runtime-only, never saved
    go.transform.position = Vector3.zero;
    bgmSrc = AddSource(go, FluxAmbientBed.BgmVolume);
    noiseSrc = AddSource(go, FluxAmbientBed.AmbientVolume);
    weatherSrc = AddSource(go, FluxAmbientBed.AmbientVolume);
    if (noiseBed != null)   // always-on city noise floor starts with the bed
    {
        noiseSrc.clip = noiseBed;
        noiseSrc.Play();
    }
}
static AudioSource AddSource(GameObject go, float vol)
{
    AudioSource s = go.AddComponent&lt;AudioSource&gt;();
    s.playOnAwake = false;
    s.loop = true;
    s.volume = vol;
    return s;
}
void SetLoop(AudioSource s, string assetPath, float vol)
{
    if (s == null) return;
```

```
            if (string.IsNullOrEmpty(assetPath)) { s.Stop(); s.clip = null; return; }
            AudioClip c = ResolveClip(assetPath);
            if (c == null) { s.Stop(); s.clip = null; return; }
            s.clip = c;
            s.volume = vol;
            s.Play();
        }
    }
}
// CityBubbles.cs
// FluxVerse P-23(2) r42 / r144: CityBubbles - thin MonoBehaviour adapter of
// the street bubble layer. SEPARATE FILE LAW (r14): the component class must
// live in &lt;ClassName&gt;.cs or the saved scene reference dies across editor
// sessions. The pure faces stay split: ResidentBarks (r41 data core:
// pool/pick/fact gate/budget) + ResidentBubbleRules (r42 mounting law) +
// MoodDirector (r144 mood mirror). This adapter only WIRES: poll
// world/world-state.json + world/world-events.jsonl READ-ONLY every ~10s
// (perceptor owns all writes), derive the context through the draw.py fact
// gate (events tail &gt; weather &gt; clock), then on the CLOCK tier apply the
// city mood director (mood = f(date, time, 24h event density); each
// budgeted speaker rolls its own use-ctx through the weighted lottery,
// draw.py L301-303 law - fact-gate contexts never move), budget the
// speakers (&lt;=2 on screen), then mount their bubbles above the nameplates.
//
// RUNTIME-ONLY LAW (r13/r25): every bubble GO and every loaded texture is a
// runtime child of this component - NEVER saved into the scene. The proof
// gates zero persisted BarkBubble* objects after save and reload. Bubbles are
// rebuilt only when the (date, ctx, slot) triple changes (draw.py slot law:
// 45-minute slots, 32/day) - the swap is the real clock speaking, not a
// decorative loop (DESIGN section 7 honesty law).
//
// OVERLAP POLICY (attention rationing, deterministic): BudgetedSpeakers ranks
// the roster by md5 and yields at most MaxBubblesPerScreen ids in rank order;
// the adapter mounts each in order and DROPS any later speaker whose bubble
// rect would strictly overlap an already-mounted one (widest legal line is
// 13 u - two far-apart residents may still be picked together, two neighbors
// may not; rank head always wins, the tail yields). No invented fallback.
//
// Honesty degrade (probe contract): missing BubbleData texture, missing/broken
// barks file, unreadable world state -&gt; that face stays silent; the PROOF
// fails loud instead. Pure 2D (flat sprites, ortho camera). ASCII. No 3D.
using System;
using System.Collections.Generic;
using System.IO;
using System.Text.RegularExpressions;
using UnityEngine;
namespace FluxVerse
{
    public class CityBubbles : MonoBehaviour
    {
```

```
public const float PollIntervalSec = 10f;
const int TailEvents = 15;   // draw.py law: the fact gate reads the last 15 stream lines
static readonly Regex TypeRx = new Regex("\"type\":\"([A-Z_]+)\"");
static readonly Regex TsRx = new Regex("\"ts_utc\":\"([^\"]+)\"");
[Serializable] class StateFile { public Reality reality; }
[Serializable] class Reality { public string beijing_hhmm, weather_kind; }
readonly List&lt;GameObject&gt; mounted = new List&lt;GameObject&gt;();
readonly List&lt;string&gt; mountedIds = new List&lt;string&gt;();
readonly List&lt;string&gt; mountedLines = new List&lt;string&gt;();
readonly List&lt;Rect&gt; mountedRects = new List&lt;Rect&gt;();
readonly Dictionary&lt;string, Sprite&gt; spriteCache = new Dictionary&lt;string, Sprite&gt;();
string lastDate, lastCtx, lastMood;
int lastSlot = -1;
float pollTimer = 999f;      // poll on first Update
public int BubbleCount { get { return mounted.Count; } }
public string CurrentCtx { get { return lastCtx; } }
void Update()
{
    pollTimer += Time.deltaTime;
    if (pollTimer &gt;= PollIntervalSec) { pollTimer = 0f; PollFromWorld(); }
}
// reality link: world-state (Beijing clock + weather) + events stream -&gt;
// draw.py fact gate -&gt; ctx (+ src); on the clock tier the mood director
// (24h density + alert + calendar) may re-roll each speaker's bucket.
public void PollFromWorld()
{
    try
    {
        string repo = Path.GetDirectoryName(Path.GetDirectoryName(Application.dataPath));
        StateFile sf = JsonUtility.FromJson&lt;StateFile&gt;(
            File.ReadAllText(Path.Combine(repo, "world", "world-state.json")));
        if (sf == null || sf.reality == null) return;
        string hhmm = sf.reality.beijing_hhmm;
        if (string.IsNullOrEmpty(hhmm)) return;
        int h, m;
        if (!ParseHhmm(hhmm, out h, out m)) return;
        string[] types; int density; bool alert;
        ReadStream(Path.Combine(repo, "world", "world-events.jsonl"), out types, out density, out alert);
        DateTime bj = DateTime.Now.Date + new TimeSpan(h, m, 0);
        string src;
        string ctx = ResidentBarks.DeriveContext(types, sf.reality.weather_kind, bj, out src);
        MoodState mood = null;
        if (src == MoodDirector.SrcClock)
            mood = MoodDirector.DeriveState(bj, density, MoodDirector.DenseDefault, alert,
                sf.reality.weather_kind, MoodDirector.LoadCalendar());
        Refresh(DateTime.Now.ToString("yyyy-MM-dd"), ctx, (h * 60 + m) / 45, mood);
    }
    catch (Exception) { /* keep current bubbles: probe contract silent degrade */ }
}
// draw.py barks-tier mood seed law (L301-303 with slot=None):
```

```
// Position law (single source of truth): the tag DERIVES from ResidentRules -
// same x, y = resident y + OffsetY where OffsetY = half the resident (the
// 1.333u body top) + GapFromHead + half the tag. The 8px-above-the-head
// baseline (city-core-design sec.8) is expressed in the pack's own 24px/u
// density: 8/24 = 0.3333u.
//
// Size law: b1 uniform canvas 66x20 px at PPU 24 -&gt; 2.75x0.833 world units
// (r97: the longest registered CJK name runs 57px + margin; registered names
// are never cut - honesty law). The 2.75u plate width sets the same-row seat
// spacing law (&gt;= 2.75u x-gap) the r99 seat harness enforces.
// Colors: baked pixels are the P-18 moonlight annotation channel - AMBIENT
// scenery, five-color law untouched. ASCII. Pure 2D. No 3D.
using UnityEngine;
namespace FluxVerse
{
    public static class ResidentTagRules
    {
        public const int Order = 7;            // street layer: signs 6 &lt; street 7 &lt; tint 8
        public const float PPU = 24f;           // residents' own density divisor (r37 law)
        public const int Count = 32;            // == ResidentRules.Count
        public const string NamePrefix = "NameTag";
        public const int PxW = 66;              // b1 uniform canvas (r97 bake)
        public const int PxH = 20;
        // sec.8 nameplate-baseline law: the tag bottom hovers a CONSTANT 8px
        // above the head (this pack's 24px/u density -&gt; 0.3333u).
        public const float GapFromHead = 8f / 24f;
        // body top (0.6667u) + gap + half tag height
        public const float OffsetY = ResidentRules.HalfSide + GapFromHead + PxH / (2f * PPU);   // 1.4167
        public static string Name(int i) { return NamePrefix + i.ToString("00"); }
        // plate file by the roster's atlas-row plateIndex (proof resolves via
        // ResidentIdentity.At(i).plateIndex; 0..31 unique - gated in proof)
        public static string PlatePath(int plateIndex)
        {
            return "Assets/ArtPacks/residents-crowd/nameplates/plate-res-b1-"
                + plateIndex.ToString("00") + ".png";
        }
        public static Vector2 Pos(int i)
        {
            Vector2 r = ResidentRules.Pos(i);
            return new Vector2(r.x, r.y + OffsetY);
        }
        public static float WorldW { get { return PxW / PPU; } }   // 2.75u
        public static float WorldH { get { return PxH / PPU; } }   // 0.8333u
        // framing: every tag fully inside the L0 view (ortho 20, aspect 16:9)
        // with margin, and inside the tint band y -16..+14 (r22 debt family)
        public static bool InView(int i, float halfW, float halfH)
        {
            Vector2 p = Pos(i);
            return Mathf.Abs(p.x) + WorldW / 2f &lt;= halfW - 0.3f
                &amp;&amp; Mathf.Abs(p.y) + WorldH / 2f &lt;= halfH - 0.3f;
```

```
        }
        public static bool InTintBand(int i)
        {
            Vector2 p = Pos(i);
            return p.y - WorldH / 2f &gt;= -16f + 0.1f &amp;&amp; p.y + WorldH / 2f &lt;= 14f;
        }
    }
}
// RimLightRules.cs
// FluxVerse P-38(1) r148 (dusk rim-light layer): pure core - the engine mirror
// of Tools/city/rimlight-manifest.json (the r147 sandbox, 133 assertions green;
// that manifest = the single parameter source for THIS round, LabsRules
// manifest-mirror idiom). First aesthetics-upgrade slice: the art-target-dusk
// hero reads as building TOP edges lit by the horizon glow BEHIND the city
// (r147 first-hand correction: the rim is CENTRIPETAL - it faces the sunset
// band behind the QUANT twist tower, NOT a uniform west side; strength falls
// off with horizontal distance from that band; the outermost buildings fade
// to near-nothing). v0 = roofline strips only (the target image's most stable,
// brightest rim position); vertical glow-facing corner strips are reserved v1.
//   laws mirrored from the manifest (fluxverse-rimlight/0.1):
//   - containment: every roof segment = the top 2 art px INSIDE its building
//     rect - zero floating pixels by construction (r147 sandbox A3).
//   - pixel-art edge: hard top row + 1-step dither decay, no blur, no halo.
//   - tier gate: dusk-only v0 (dawn rim reserved); every other tier = alpha 0.
//   - color: honey orange = AMBIENT channel (dusk tint family), five-color
//     functional law untouched; segments within 1.5u of the glow center take
//     the brighter core color (QUANT roof + brain wedge tip + both shoulders).
//   - render order 5: first free slot above the tilemaps (0..4), below the
//     signs (6) / tint (8) / band (9) / pulses (10) - the CEO pulse stays
//     above the rim (CEO-precedence law, r146 mood gate).
//   - r146 input-clamp note: a multiplicative tint can only dim, so the rim
//     is a NORMAL alpha-blended bright color, never an RGB&gt;1 tint.
// Proof surface: RimLightProof A cross-checks THIS mirror against the
// manifest JSON; RimLightProof B/C gate the wiring census + tier law;
// RimLightProof D gates the render differential census (dusk on-vs-off per
// segment, day/night zero-delta); reload gate re-verifies across a restart.
// ASCII. Pure static core, no MonoBehaviour. No 3D.
using UnityEngine;
namespace FluxVerse
{
    public static class RimLightRules
    {
        public const string Protocol = "fluxverse-rimlight/0.1";
        public const int BakedRound = 147;              // manifest origin (single parameter source)
        public const int SegmentCount = 11;
        // render.order: free slot above tilemaps 0..4, below signs 6 / street 7 /
        // tint 8 / band 9 / pulses 10 / drops 12 (CEO pulse 10 stays above = law)
        public const int SortOrder = 5;
        // art grid: 1 art px = 1/16 u (manifest rect coordinates snap to this grid)
        public const float ArtPx = 1f / 16f;
```

```
public const int RoofArtPx = 2;                // thickness.roof_art_px
public const float ThicknessU = 0.125f;         // thickness.world_u (2 art px)
public const int DitherRows = 2;                // dither.art_rows length
// dither.art_rows: [0] = hard TOP row, [1] = 1-step decay below it
public static float DitherRowAlpha(int rowFromTop)
{
    if (rowFromTop == 0) return 0.80f;
    return 0.38f;
}
// glow.center_x: sunset horizon band behind the QUANT tower (hero position)
public const float GlowCenterX = 0.5f;
// color.* (hex refs #FFBE7D main / #FFD9A0 core) - AMBIENT channel
public static readonly Color MainColor = new Color(1.0f, 0.745f, 0.49f, 1f);
public static readonly Color CoreColor = new Color(1.0f, 0.851f, 0.627f, 1f);
public const float CoreMaxDist = 1.5f;          // color.core_max_dist
// falloff.bands: closed cover, first band whose max_dist covers wins
public const int FalloffBandCount = 3;
public static float BandMaxDist(int b)
{
    if (b == 0) return 10f;
    if (b == 1) return 24f;
    return 99f;
}
public static float BandStrength(int b)
{
    if (b == 0) return 1.0f;
    if (b == 1) return 0.55f;
    return 0.25f;
}
// segments (manifest order; rect = [x0, y0, x1, y1] world u, y1-y0 = 2 art px)
static readonly string[] Ids = {
    "RimQuantTop",        // QUANT twist-tower roof (glow-center hero, core color)
    "RimGameMainTop",     // GAME_MAIN tower roof
    "RimGameAnnexTop",    // GAME_ANNEX low block roof (builder Block, off the sign table)
    "RimMediaTop",        // MEDIA roofline (full-width Block strip, silhouette-faithful)
    "RimNWTop",           // north-west low roof
    "RimNWMidTop",        // north-west-mid low roof
    "RimNEMidTop",        // north-east-mid low roof
    "RimNETop",          // north-east low roof
    "RimBrainTip",        // brain tower wedge tip (row 18 top at 19)
    "RimBrainShoulderW",  // shoulder row 17 exposed west of the tip footprint
    "RimBrainShoulderE"   // shoulder row 17 exposed east of the tip footprint
};
static readonly float[,] Rects = {
    { -2.0f, -8.125f,   3.0f, -8.0f  },
    { -23.0f, -11.125f, -18.0f, -11.0f },
    { -26.0f, -13.125f, -25.0f, -13.0f },
    { 19.0f, -8.125f,  25.0f, -8.0f  },
    { -28.0f, 11.875f, -25.0f, 12.0f  },
    { -9.0f, 11.875f,  -6.0f, 12.0f  },
```

```
            { 7.0f, 11.875f,  10.0f, 12.0f  },
            { 26.0f, 11.875f,  29.0f, 12.0f  },
            { 0.0f, 18.875f,   1.0f, 19.0f  },
            { -1.0f, 17.875f,   0.0f, 18.0f  },
            { 1.0f, 17.875f,   2.0f, 18.0f  }
        };
        public static string Id(int i) { return Ids[i]; }
        public static float X0(int i) { return Rects[i, 0]; }
        public static float Y0(int i) { return Rects[i, 1]; }
        public static float X1(int i) { return Rects[i, 2]; }
        public static float Y1(int i) { return Rects[i, 3]; }
        public static float CenterX(int i) { return (Rects[i, 0] + Rects[i, 2]) * 0.5f; }
        public static float WidthU(int i) { return Rects[i, 2] - Rects[i, 0]; }
        public static int ArtPxWidth(int i) { return Mathf.RoundToInt(WidthU(i) / ArtPx); }
        // falloff.metric = abs(segment_center_x - glow.center_x)
        public static float DistFor(int i) { return Mathf.Abs(CenterX(i) - GlowCenterX); }
        public static float StrengthForDist(float dist)
        {
            for (int b = 0; b &lt; FalloffBandCount; b++)
                if (dist &lt;= BandMaxDist(b)) return BandStrength(b);
            return BandStrength(FalloffBandCount - 1);
        }
        public static float StrengthFor(int i) { return StrengthForDist(DistFor(i)); }
        // core law: near the glow center the rim takes the brighter core color
        public static bool IsCore(int i) { return DistFor(i) &lt;= CoreMaxDist; }
        public static Color BaseColorFor(int i) { return IsCore(i) ? CoreColor : MainColor; }
        // tier_gate.tiers == ["dusk"] (dusk-only v0; dawn reserved)
        public static bool TierShows(AmbientTier t) { return t == AmbientTier.Dusk; }
        // renderer alpha for segment i: the falloff strength when the tier shows
        // rims, hard zero otherwise (instant switch = ApplyAmbient family law).
        // The per-art-row dither alphas live in the texture; this alpha carries
        // the strength, so effective top-row alpha = dither[0] x strength.
        public static float AlphaFor(int i, AmbientTier t)
        {
            return TierShows(t) ? StrengthFor(i) : 0f;
        }
    }
}
// RobotRules.cs
// FluxVerse P-28 remaining wiring face (r36): street-robot pure rules (source pack =
// tophat-robot, CC-BY 3.0 Nelson Yiap, ledger City/Assets/ArtPacks/ARTPACKS-LEDGER.md,
// credits line already in the ledger header). DESIGN section 2 organ table: robots
// walking the streets = the fleet's AI labor made visible ("blood cells = street
// robots"); P-17 item 3 pins the spec at 16x16 = exactly this pack. v0 = STATIC
// standing scenery persisted in the scene like the tilemaps and the neon signs
// (r35 static-vs-transient boundary); the walking animation itself is an M2 face
// (P-12 event sources / P-22 census identity pool), never a decorative loop.
// Frame inventory (sheet cut by Tools/city/crop-robot-frames.ps1, 16 slots): F00/F08
// standing, F05/F07 leg-together, F01/F04/F06/F12 stride steps; 8 slots are EMPTY
// (the author's mirror column was never filled - all poses face right). Each robot
```

```
// mounts a DISTINCT usable frame = the r35 free-variety law (no clone row).
// Placement domain (CitySkeletonBuilder geometry): south QUANT plaza pavement rows
// -16..-9, south street rows -8..-7 and north street rows 7..8 (roads layer), north
// promenade row 4; the proof re-derives "stands on a Ground/Roads tile cell" from
// the live tilemaps instead of trusting this comment. Sorting law: street layer
// order 7 = above mounting signs 6, below the ambient tint 8. Scale law: native
// PPU16, scale 1 -&gt; zero resampling (r34 importer law enforced in the proof).
// Five-color law untouched: pack teal/gray/orange is ambient scenery. ASCII. No 3D.
using UnityEngine;
namespace FluxVerse
{
    public static class RobotRules
    {
        public const int Order = 7;            // street layer: signs 6 &lt; robots 7 &lt; tint 8
        public const float PPU = 16f;           // world law (r13); importer enforced, never assumed
        public const int Count = 8;
        public const string NamePrefix = "Robot";
        public const int PxSide = 16;           // P-17 item 3: 16x16 street robot
        // grounding-shadow constants (r87 P-69 slice 3, sec.8 check 3). Robots KEEP
        // their street-walking domain (DESIGN organ table: street robots = fleet
        // labor; roads stay legal standing ground for them, only residents are
        // sidewalk-bound) - the shadow is the grounding the r34/r45 casts lacked.
        public const string ShadowNamePrefix = "ShadowBot";
        public const int ShadowOrder = 5;        // Props 4 &lt; shadows 5 &lt; signs 6
        public const int ShadowPxW = 16;          // 16x6 @ PPU16 = 1.0 x 0.375u native
        public const int ShadowPxH = 6;
        public const string ShadowPath =
            "Assets/ArtPacks/tophat-robot/shadows/shadow-bot.png";
        public const float ShadowDropY = 0.06f;  // shadow center sits this far BELOW the feet line
        public static string ShadowName(int i) { return ShadowNamePrefix + i.ToString("00"); }
        public static Vector2 ShadowPos(int i)
        {
            Bot b = Table[i];
            return new Vector2(b.x, b.y - PxSide / (PPU * 2f) - ShadowDropY);   // feet at y-0.5
        }
        public static float ShadowWorldW { get { return ShadowPxW / PPU; } }
        public static float ShadowWorldH { get { return ShadowPxH / PPU; } }
        public struct Bot
        {
            public string name;    // scene GO name (ASCII)
            public string path;    // project-relative asset path (Assets/...)
            public int frame;      // sheet frame index (contact sheet F##, reporting only)
            public float x, y;     // world center at scale 1 (sprite pivot = center)
        }
        public static Bot At(int i) { return Table[i]; }
        public static string Name(int i) { return Table[i].name; }
        public static string Path(int i) { return Table[i].path; }
        public static Vector2 Pos(int i) { return new Vector2(Table[i].x, Table[i].y); }
        public static float WorldW(int i) { return PxSide / PPU; }
        public static float WorldH(int i) { return PxSide / PPU; }
```

```
        // framing: every robot fully inside the L0 view (ortho 20, aspect 16:9) with
        // margin, and inside the tint band y -16..+14 (an untinted sliver would read
        // as the r22 edge-band debt all over again)
        public static bool InView(int i, float halfW, float halfH)
        {
            Bot b = Table[i];
            float hw = PxSide / (PPU * 2f), hh = PxSide / (PPU * 2f);
            return Mathf.Abs(b.x) + hw &lt;= halfW - 0.3f &amp;&amp; Mathf.Abs(b.y) + hh &lt;= halfH - 0.3f;
        }
        public static bool InTintBand(int i)
        {
            Bot b = Table[i];
            float hh = PxSide / (PPU * 2f);
            return b.y - hh &gt;= -16f + 0.1f &amp;&amp; b.y + hh &lt;= 14f;
        }
        // a world point stands on a walkable cell only if the Ground (pavement) or
        // Roads tilemap holds a tile there - water, roofs, building interiors and
        // empty sky all fail. Pure math half (cell derivation); the tilemap lookup
        // lives in the proof (editor face).
        public static Vector2Int CellOf(int i)
        {
            Bot b = Table[i];
            return new Vector2Int(Mathf.FloorToInt(b.x), Mathf.FloorToInt(b.y));
        }
        static readonly Bot[] Table = new Bot[]
        {
            // 0. south QUANT plaza, west side - standing idle frame
            new Bot { name = "RobotPlazaW",  path = "Assets/ArtPacks/tophat-robot/frames/robot_f00.png", frame = 0,  x =
            // 1. south QUANT plaza, east side - standing idle twin frame
            new Bot { name = "RobotPlazaE",  path = "Assets/ArtPacks/tophat-robot/frames/robot_f08.png", frame = 8,  x =
            // 2. south QUANT plaza, tower front center - leg-together pose
            //    (r104: SW of the new tower footprint; tower front center is now wall)
            new Bot { name = "RobotPlazaS",  path = "Assets/ArtPacks/tophat-robot/frames/robot_f05.png", frame = 5,  x =
            // 3. south street west stretch (between GAME and QUANT) - stride step
            new Bot { name = "RobotStWest",  path = "Assets/ArtPacks/tophat-robot/frames/robot_f04.png", frame = 4,  x =
            // 4. south street east stretch (between QUANT and MEDIA) - opposite-phase stride
            new Bot { name = "RobotStEast",  path = "Assets/ArtPacks/tophat-robot/frames/robot_f06.png", frame = 6,  x =
            // 5. GAME city front plaza - leg-together pose
            //    (r104: GAME_MAIN north face row - the old front plaza cell is now
            //    inside the tower; street robots keep the road-legal stand domain)
            new Bot { name = "RobotGameFr",  path = "Assets/ArtPacks/tophat-robot/frames/robot_f07.png", frame = 7,  x =
            // 6. MEDIA city front plaza - stride step
            //    (r104: west of the new dual-sphere footprint, vcol road cell)
            new Bot { name = "RobotMediaFr", path = "Assets/ArtPacks/tophat-robot/frames/robot_f01.png", frame = 1,  x =
            // 7. north promenade east (pavement row 4) - wide stride
            new Bot { name = "RobotPromE",   path = "Assets/ArtPacks/tophat-robot/frames/robot_f12.png", frame = 12, x =
        };
    }
}
// StreetBehaviorRules.cs
```

```
// FluxVerse P-75 slice B (r124): street-resident behavior consumption - PURE LAW CORE.
// Sources (single-source law): TECH sec.9 r122 row (the consumption law) +
// Tools/city/eaveslots-manifest.json (r123 sandbox verdict, 365 assertions - the
// eave geometry single source; ResidentProof cross-checks the table below
// against that manifest file every run, OfficeProof r111 pattern).
//
// Data face: world/world-state.json -&gt; state.street_behavior (probe
// street_behavior r121): { ctx, generated_utc, total, visible, nodata,
// seats[32] }, per seat { slot, go, id, state, visible, quirk, recovering }.
// The world/ dir is the perceptor's single-writer domain (P-43 law): this core
// READS ONLY, never writes. The seats are roster-ordered street slots - the
// engine slot index IS the seat index (ResidentRules table).
//
// LAWS (r122 item 1, mirrored verbatim):
//  (b) VISIBILITY - per-seat presence = (visible == 1). FULL re-derive on every
//      apply (no incremental drift - the R3 "same input, same bytes" law's
//      engine-side dual: position and presence are ALWAYS state-derived).
//      Absent section / absent file = GRANDFATHER: all 32 visible, all home
//      (zero regression while BigLife's export is missing). A row whose go
//      name does not match the seat keeps grandfather for that seat (honest
//      degrade on data drift - the coupling key is the GO name).
//  (c) SHELTER - state == "shelter" AND quirk == "umbrella" -&gt; stays at the
//      street HOME position (the street-with-umbrella face, behavior.py law 3);
//      state == "shelter" without umbrella AND a deterministic bucket hit -&gt;
//      candidate for the district eave slot. Capacity 1 per slot: the candidate
//      with the LOWEST roster slot takes it, overflow stays home (r123
//      assignment law). Districts with no slot (QUANT / GAME), the TOWER anchor
//      seat and cross-city VISITOR seats fall back home (honest fallback - no
//      fake shelter for a district with no eave). Any other state restores
//      home. Invisible shelterers stay home (presence law wins).
//  (d) ANCHOR - NO special case: the C-00001 honor seat follows the data face
//      like everyone else (r122 verdict - a permanent-presence override would
//      be decorative and break the honesty law; a CEO flip is one line here).
//
// BUCKET HASH (this core's own law - r122 left the function to the engine
// round): FNV-1a (32-bit offset basis 2166136261, prime 16777619) over the
// census id characters, h % 10 &lt; 3 -&gt; a ~30% deterministic subset. Current
// roster census: 9/32 hits (MEDIA 4: slots 17/18/20/23 - lowest takes the
// slot; NORTH 1: slot 25). behavior.py decides WHO is visible / sheltering;
// this core only derives presence + position. ASCII. Pure 2D. No 3D.
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;
namespace FluxVerse
{
    // one street seat's behavior row (mirrors the probe state face)
    [Serializable]
    public class StreetBehaviorSeat
    {
```

```
    public int slot;
    public string go;
    public string id;
    public string state;
    public int visible;
    public string quirk;
    public int recovering;
}
// the whole street_behavior state section
[Serializable]
public class StreetBehaviorSnapshot
{
    public string ctx;
    public string generated_utc;
    public int total;
    public int visible;
    public int nodata;
    public StreetBehaviorSeat[] seats;
}
// pure static law core: parse + derive. No MonoBehaviour.
public static class StreetBehaviorRules
{
    public const int Count = 32;                    // == ResidentRules.Count
    public const string ShelterState = "shelter";     // behavior.py STATES member
    public const string UmbrellaQuirk = "umbrella";  // behavior.py QUIRKS member
    public const int BucketMod = 10;
    public const int BucketCut = 3;                   // h % 10 &lt; 3
    // ---- eave slots (r123 manifest: quota MEDIA 1 + NORTH 1, capacity 1) ----
    public struct EaveSlot
    {
        public string id;      // manifest slot id
        public string district; // seat ZONE the slot serves
        public float x, y;     // stand position (seat-class law: body 1.333u)
        public int capacity;
    }
    public const int EaveCount = 2;
    static readonly EaveSlot[] EaveTable = new EaveSlot[]
    {
        new EaveSlot { id = "EAVE-M01", district = "MEDIA", x = 33.5f,    y = -11f, capacity = 1 },
        new EaveSlot { id = "EAVE-N01", district = "NORTH", x = 24.6667f, y = 11f,  capacity = 1 },
    };
    public static EaveSlot Eave(int e) { return EaveTable[e]; }
    public static Vector2 EavePos(int e) { return new Vector2(EaveTable[e].x, EaveTable[e].y); }
    // seat zone -&gt; eave slot index; -1 = no slot (honest home fallback)
    public static int EaveIndexForZone(string zone)
    {
        for (int e = 0; e &lt; EaveCount; e++)
            if (EaveTable[e].district == zone) return e;
        return -1;
    }
```

```
// deterministic bucket hash: FNV-1a over the census id, reduced mod 10
public static int HashOf(string id)
{
    if (string.IsNullOrEmpty(id)) return 0;
    uint h = 2166136261u;
    for (int k = 0; k &lt; id.Length; k++)
    {
        h ^= (uint)id[k];
        h *= 16777619u;
    }
    return (int)(h % (uint)BucketMod);
}
public static bool BucketHit(string id) { return HashOf(id) &lt; BucketCut; }
[Serializable]
class StateFile { public StreetBehaviorSnapshot street_behavior; }
// whole-state-file parse (unknown fields ignored); null = honest absence
public static StreetBehaviorSnapshot Parse(string json)
{
    if (string.IsNullOrEmpty(json)) return null;
    try
    {
        StateFile f = JsonUtility.FromJson&lt;StateFile&gt;(json);
        return f == null ? null : f.street_behavior;
    }
    catch { return null; }
}
static string StatePath()
{
    string projectRoot = Path.GetDirectoryName(Application.dataPath);   // .../City
    string repoRoot = Path.GetDirectoryName(projectRoot);               // .../FluxVerse
    return Path.Combine(repoRoot, "world", "world-state.json");
}
// the live perceptor snapshot; null = absent file / absent section
public static StreetBehaviorSnapshot LoadStateFile()
{
    string p = StatePath();
    if (!File.Exists(p)) return null;
    return Parse(File.ReadAllText(p));
}
// ---- derive: the one law path play mode and proofs both walk ----
public struct SeatPlan
{
    public bool visible;    // presence law (b)
    public Vector2 pos;     // home restore / eave relocate (c)
    public int eave;        // eave slot index served, -1 = home
    public bool managed;    // false = grandfather (no matching row)
}
public static void Derive(StreetBehaviorSnapshot snap, SeatPlan[] plans)
{
    if (plans == null || plans.Length != Count)
```

```
                throw new InvalidOperationException("plans array must hold exactly " + Count + " seats");
            for (int i = 0; i &lt; Count; i++)
            {
                plans[i].visible = true;
                plans[i].pos = ResidentRules.Pos(i);
                plans[i].eave = -1;
                plans[i].managed = false;
            }
            if (snap == null || snap.seats == null) return;   // grandfather face
            List&lt;int&gt;[] wait = new List&lt;int&gt;[EaveCount];
            for (int e = 0; e &lt; EaveCount; e++) wait[e] = new List&lt;int&gt;();
            for (int r = 0; r &lt; snap.seats.Length; r++)
            {
                StreetBehaviorSeat row = snap.seats[r];
                if (row == null) continue;
                int s = row.slot;
                if (s &lt; 0 || s &gt;= Count) continue;
                if (row.go != ResidentRules.Name(s)) continue;   // drift row -&gt; grandfather
                plans[s].managed = true;
                plans[s].visible = row.visible == 1;
                if (!plans[s].visible) continue;                 // presence law wins
                if (row.state != ShelterState) continue;        // non-shelter restores home
                if (row.quirk == UmbrellaQuirk) continue;       // street-with-umbrella face
                if (!BucketHit(row.id)) continue;               // bucket miss stays home
                int e = EaveIndexForZone(ResidentRules.ZoneOf(s));
                if (e &gt;= 0) wait[e].Add(s);                       // no slot / visitor / tower -&gt; home
            }
            for (int e = 0; e &lt; EaveCount; e++)
            {
                if (wait[e].Count == 0) continue;
                int best = wait[e][0];                          // capacity pick: lowest roster slot
                for (int k = 1; k &lt; wait[e].Count; k++)
                    if (wait[e][k] &lt; best) best = wait[e][k];
                plans[best].pos = EavePos(e);
                plans[best].eave = e;
            }
        }
    }
}
// TDDemo.cs
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Tilemaps;
// TD greybox demo - runtime movement layer (CEO direct order 2026-09-28).
// Walkers: BFS paths over road cells, deterministic seeded target picking.
// Player: WASD/arrow free walk, camera follows with map-bounds clamp.
public static class TDWalkLib
{
    public const int MapSize = 64;
    public static HashSet&lt;Vector3Int&gt; CollectWalkable(Tilemap roads)
```

```
{
    var set = new HashSet&lt;Vector3Int&gt;();
    if (roads == null) return set;
    var b = roads.cellBounds;
    int cells = 0;
    foreach (var c in b.allPositionsWithin)
    {
        cells++;
        if (cells &gt; 50000) break; // r126-style guard
        if (roads.GetTile(c) != null) set.Add(c);
    }
    return set;
}
// tileless mode (anime v2 organic street network, CEO order 2026-09-28 ~14:xx)
public static HashSet&lt;Vector3Int&gt; RoadCells()
{
    var set = new HashSet&lt;Vector3Int&gt;();
    const string DATA = "0,33 0,34 0,35 1,33 1,34 1,35 1,36 2,34 2,35 2,36 3,31 3,34 3,35 3,36 3,37 4,30 4,31 4,32 4
    foreach (var tok in DATA.Split(' '))
    {
        var p = tok.Split(',');
        set.Add(new Vector3Int(int.Parse(p[0]), int.Parse(p[1]), 0));
    }
    return set;
}
public static List&lt;Vector3Int&gt; Path(HashSet&lt;Vector3Int&gt; walkable, Vector3Int from, Vector3Int to)
{
    var result = new List&lt;Vector3Int&gt;();
    if (walkable == null || walkable.Count == 0) return result;
    if (!walkable.Contains(from) || !walkable.Contains(to)) return result;
    var prev = new Dictionary&lt;Vector3Int, Vector3Int&gt;();
    var q = new Queue&lt;Vector3Int&gt;();
    q.Enqueue(from);
    prev[from] = from;
    int[] dx = { 1, -1, 0, 0 };
    int[] dy = { 0, 0, 1, -1 };
    while (q.Count &gt; 0)
    {
        var cur = q.Dequeue();
        if (cur == to) break;
        for (int i = 0; i &lt; 4; i++)
        {
            var n = new Vector3Int(cur.x + dx[i], cur.y + dy[i], 0);
            if (walkable.Contains(n) &amp;&amp; !prev.ContainsKey(n))
            {
                prev[n] = cur;
                q.Enqueue(n);
            }
        }
    }
```

```
        if (!prev.ContainsKey(to)) return result;
        var node = to;
        while (node != from)
        {
            result.Add(node);
            node = prev[node];
        }
        result.Add(from);
        result.Reverse();
        return result;
    }
    public static Vector3 CellCenter(Vector3Int c)
    {
        return new Vector3(c.x + 0.5f, c.y + 0.5f, 0f);
    }
    public static Vector3 PosAt(List&lt;Vector3Int&gt; path, float t)
    {
        if (path == null || path.Count == 0) return Vector3.zero;
        if (path.Count == 1) return CellCenter(path[0]);
        float total = path.Count - 1;
        float f = Mathf.Repeat(t, total);
        int i = Mathf.FloorToInt(f);
        float k = f - i;
        return Vector3.Lerp(CellCenter(path[i]), CellCenter(path[i + 1]), k);
    }
}
public class TDBootstrap : MonoBehaviour
{
    void Awake()
    {
        var roadsGo = GameObject.Find("TD_Roads");
        Tilemap roads = roadsGo != null ? roadsGo.GetComponent&lt;Tilemap&gt;() : null;
        var walkable = roads != null ? TDWalkLib.CollectWalkable(roads) : TDWalkLib.RoadCells();
        foreach (var w in Object.FindObjectsOfType&lt;TDWalker&gt;()) w.Init(walkable);
    }
}
public class TDWalker : MonoBehaviour
{
    public Vector3Int startCell;
    public int seed = 1;
    public float speed = 2.0f;
    HashSet&lt;Vector3Int&gt; _walkable;
    List&lt;Vector3Int&gt; _path;
    float _t;
    System.Random _rng;
    SpriteRenderer _sr;
    SpriteRenderer _shadowSr;
    public void Init(HashSet&lt;Vector3Int&gt; walkable)
    {
        _walkable = walkable;
```

```
        _rng = new System.Random(seed);
        _sr = GetComponent&lt;SpriteRenderer&gt;();
        var sh = transform.Find("Shadow");
        if (sh != null) _shadowSr = sh.GetComponent&lt;SpriteRenderer&gt;();
        transform.position = TDWalkLib.CellCenter(startCell);
        NewPath();
    }
    void NewPath()
    {
        if (_walkable == null || _walkable.Count == 0) { _path = null; return; }
        var arr = new List&lt;Vector3Int&gt;(_walkable);
        Vector3Int to;
        int guard = 0;
        do { to = arr[_rng.Next(arr.Count)]; guard++; } while (to == CurrentCell() &amp;&amp; guard &lt; 20);
        _path = TDWalkLib.Path(_walkable, CurrentCell(), to);
        _t = 0f;
        if (_path == null || _path.Count &lt; 2) _path = null;
    }
    Vector3Int CurrentCell()
    {
        var p = transform.position;
        return new Vector3Int(Mathf.FloorToInt(p.x), Mathf.FloorToInt(p.y), 0);
    }
    void Update()
    {
        if (_path == null || _path.Count &lt; 2) { if (_walkable != null) NewPath(); return; }
        _t += Time.deltaTime * speed;
        transform.position = TDWalkLib.PosAt(_path, _t);
        if (_t &gt;= _path.Count - 1) NewPath();
        // Y-sort against buildings/trees (view discipline v1: pseudo depth)
        int ord = 512 - Mathf.FloorToInt(transform.position.y);
        if (_sr != null) _sr.sortingOrder = ord;
        if (_shadowSr != null) _shadowSr.sortingOrder = ord - 1;
    }
}
public class TDPlayer : MonoBehaviour
{
    public float speed = 4.0f;
    public float minZoom = 4.5f;
    float _zoomTarget = 11.25f;
    SpriteRenderer _sr;
    SpriteRenderer _shadowSr;
    void Awake()
    {
        _sr = GetComponent&lt;SpriteRenderer&gt;();
        var sh = transform.Find("Shadow");
        if (sh != null) _shadowSr = sh.GetComponent&lt;SpriteRenderer&gt;();
    }
    void Update()
    {
```

```
        // CEO order 2026-09-28: camera must zoom (pull close / push far).
        // One wheel notch = x0.88 in / x1.14 out; max zoom keeps the map
        // half-width exactly filling the screen so no void shows past edges.
        var cam = Camera.main;
        if (cam != null &amp;&amp; cam.orthographic)
        {
            float maxZoom = (TDWalkLib.MapSize * 0.5f) / cam.aspect;
            float d = Input.GetAxisRaw("Mouse ScrollWheel");
            if (d &gt; 0.001f) _zoomTarget = Mathf.Clamp(_zoomTarget * 0.88f, minZoom, maxZoom);
            else if (d &lt; -0.001f) _zoomTarget = Mathf.Clamp(_zoomTarget / 0.88f, minZoom, maxZoom);
            cam.orthographicSize = Mathf.MoveTowards(cam.orthographicSize, _zoomTarget, 24f * Time.deltaTime);
        }
        float dx = Input.GetAxisRaw("Horizontal");
        float dy = Input.GetAxisRaw("Vertical");
        var p = transform.position + new Vector3(dx, dy, 0f) * speed * Time.deltaTime;
        p.x = Mathf.Clamp(p.x, 1f, TDWalkLib.MapSize - 1f);
        p.y = Mathf.Clamp(p.y, 1f, TDWalkLib.MapSize - 1f);
        transform.position = p;
        if (cam != null &amp;&amp; cam.orthographic)
        {
            float cs = cam.orthographicSize;
            float cx = Mathf.Clamp(p.x, cs * cam.aspect, TDWalkLib.MapSize - cs * cam.aspect);
            float cy = Mathf.Clamp(p.y, cs, TDWalkLib.MapSize - cs);
            cam.transform.position = new Vector3(cx, cy, -10f);
        }
        // Y-sort against buildings/trees (view discipline v1: pseudo depth)
        int ord = 512 - Mathf.FloorToInt(p.y);
        if (_sr != null) _sr.sortingOrder = ord;
        if (_shadowSr != null) _shadowSr.sortingOrder = ord - 1;
    }
}
// UiKit.cs
// FluxVerse P-18 r22: UiKit - pure procedural builder for the GUIAgent-style UI
// shell (CEO order ~18:55, ledger P-2026-09-23-18, ruling A: UI shell adopts the
// GUIAgent display style, the city world layer keeps its pixel canon). First
// slice: glassmorphism panel + crystal button. ALL 2D light effects, no 3D:
//   Panel  = dark premium glass (moonlight LUT cool darks, semi-transparent)
//            + thin bright rim (xi-miao-bian) + soft outer glow (fa-guang)
//   Button = same glass + accent rim (warm gold = nuan-guang dian-zhui)
//            + top-left highlight streak (gao-guang) + bottom inner shadow
// Procedural SpriteRenderer stacks only - r16 law: a WorldSpace canvas renders
// as ScreenSpaceOverlay in batch mode, so batch-verified UI walks the sprite
// path (renders identically in play mode). Widgets are RUNTIME-ONLY (r13 law):
// built on demand, never saved into a scene. localScale is ALWAYS normalized
// against sprite.bounds.size (r13 law). Scripts ASCII (encoding law).
using System;
using System.Collections.Generic;
using UnityEngine;
namespace FluxVerse
{
```

```
public static class UiKit
{
    public const float PPU = 32f;
    public const float Depth = -9f;
    // moonlight palette (P-18 2: cool darks + warm accents)
    public static readonly Color GlassTop = new Color(0.06f, 0.085f, 0.13f);
    public static readonly Color GlassBottom = new Color(0.035f, 0.05f, 0.08f);
    public const float GlassAlpha = 0.78f;
    public static readonly Color CoolRim = new Color(0.72f, 0.82f, 0.96f);
    public static readonly Color GlowColor = new Color(0.55f, 0.68f, 0.95f);
    public static readonly Color WarmAccent = new Color(0.92f, 0.76f, 0.35f);
    public static readonly Color StreakColor = new Color(0.82f, 0.90f, 1f);
    // UI shell owns sorting orders 40..59 - above every world layer
    // (banner halo..text = 20..23, weather/alert lower)
    public const int PanelGlowOrder = 40, PanelGlassOrder = 41, PanelRimOrder = 42;
    public const int ButtonGlowOrder = 45, ButtonGlassOrder = 46,
                     ButtonRimOrder = 47, StreakOrder = 48, ShadowOrder = 49;
    // ---- widget builders (runtime-only, caller owns lifetime) ----
    public static GameObject BuildPanel(Transform parent, string name, Vector2 center, Vector2 size)
    {
        GameObject root = new GameObject(name);
        if (parent != null) root.transform.SetParent(parent, false);
        root.transform.position = new Vector3(center.x, center.y, Depth);
        SpriteRenderer glass = Layer(root.transform, "Glass",
            TexturedSprite(MakeGlassTex(TexW(size.x), TexH(size.y))), PanelGlassOrder);
        Fit(glass, size);
        float rimW = RimPx(size) / PPU;
        SpriteRenderer rim = Layer(root.transform, "Rim",
            TexturedSprite(MakeRimTex(TexW(size.x + 2f * rimW), TexH(size.y + 2f * rimW), RimPx(size))),
            PanelRimOrder);
        rim.color = CoolRim;
        Fit(rim, size + new Vector2(2f * rimW, 2f * rimW));
        float range = GlowRange(size);
        SpriteRenderer glow = Layer(root.transform, "Glow",
            TexturedSprite(MakeGlowTex(TexW(size.x + 2f * range), TexH(size.y + 2f * range), range * PPU)),
            PanelGlowOrder);
        glow.color = GlowColor;
        Fit(glow, size + new Vector2(2f * range, 2f * range));
        return root;
    }
    // P-18 slice 2 (r23): the P-16 interior banner upgrades to this same
    // recipe, but keeps the banner sorting band (20..23) instead of the UI
    // shell band (40..59) - callers own the orders. Any world-layer glass
    // widget that must render below the UI shell uses this entry point.
    public static GameObject BuildGlassPanel(Transform parent, string name, Vector2 center,
        Vector2 size, int glowOrder, int glassOrder, int rimOrder)
    {
        GameObject root = new GameObject(name);
        if (parent != null) root.transform.SetParent(parent, false);
        root.transform.position = new Vector3(center.x, center.y, Depth);
```

```
        SpriteRenderer glass = Layer(root.transform, "Glass",
            TexturedSprite(MakeGlassTex(TexW(size.x), TexH(size.y))), glassOrder);
        Fit(glass, size);
        float rimW = RimPx(size) / PPU;
        SpriteRenderer rim = Layer(root.transform, "Rim",
            TexturedSprite(MakeRimTex(TexW(size.x + 2f * rimW), TexH(size.y + 2f * rimW), RimPx(size))),
            rimOrder);
        rim.color = CoolRim;
        Fit(rim, size + new Vector2(2f * rimW, 2f * rimW));
        float range = GlowRange(size);
        SpriteRenderer glow = Layer(root.transform, "Glow",
            TexturedSprite(MakeGlowTex(TexW(size.x + 2f * range), TexH(size.y + 2f * range), range * PPU)),
            glowOrder);
        glow.color = GlowColor;
        Fit(glow, size + new Vector2(2f * range, 2f * range));
        return root;
    }
    public static GameObject BuildButton(Transform parent, string name, Vector2 center, Vector2 size, Color accent)
    {
        GameObject root = new GameObject(name);
        if (parent != null) root.transform.SetParent(parent, false);
        root.transform.position = new Vector3(center.x, center.y, Depth);
        SpriteRenderer glass = Layer(root.transform, "Glass",
            TexturedSprite(MakeGlassTex(TexW(size.x), TexH(size.y))), ButtonGlassOrder);
        Fit(glass, size);
        float rimW = RimPx(size) / PPU;
        SpriteRenderer rim = Layer(root.transform, "Rim",
            TexturedSprite(MakeRimTex(TexW(size.x + 2f * rimW), TexH(size.y + 2f * rimW), RimPx(size))),
            ButtonRimOrder);
        rim.color = accent;
        Fit(rim, size + new Vector2(2f * rimW, 2f * rimW));
        SpriteRenderer streak = Layer(root.transform, "Streak",
            TexturedSprite(MakeStreakTex(TexW(size.x), TexH(size.y))), StreakOrder);
        streak.color = StreakColor;
        Fit(streak, size);
        SpriteRenderer shadow = Layer(root.transform, "Shadow",
            TexturedSprite(MakeShadowTex(TexW(size.x), TexH(size.y))), ShadowOrder);
        shadow.color = new Color(0f, 0f, 0f, 1f);
        Fit(shadow, size);
        float range = GlowRange(size);
        SpriteRenderer glow = Layer(root.transform, "Glow",
            TexturedSprite(MakeGlowTex(TexW(size.x + 2f * range), TexH(size.y + 2f * range), range * PPU)),
            ButtonGlowOrder);
        glow.color = GlowColor;
        Fit(glow, size + new Vector2(2f * range, 2f * range));
        return root;
    }
    // destroy a widget root and every runtime texture it owns
    public static void Release(GameObject root)
    {
```

```
        if (root == null) return;
        List&lt;Texture2D&gt; texs = new List&lt;Texture2D&gt;();
        SpriteRenderer[] srs = root.GetComponentsInChildren&lt;SpriteRenderer&gt;(true);
        for (int i = 0; i &lt; srs.Length; i++)
            if (srs[i].sprite != null &amp;&amp; srs[i].sprite.texture != null)
                texs.Add(srs[i].sprite.texture);
        UnityEngine.Object.DestroyImmediate(root);
        for (int i = 0; i &lt; texs.Count; i++)
            if (texs[i] != null) UnityEngine.Object.DestroyImmediate(texs[i]);
    }
    // ---- procedural textures (pure, deterministic, proof-testable) ----
    public static Texture2D MakeGlassTex(int w, int h)
    {
        Texture2D t = NewTex(w, h, FilterMode.Point);
        for (int y = 0; y &lt; h; y++)
        {
            float k = h &gt; 1 ? (float)y / (h - 1) : 1f;      // y=0 bottom
            Color c = Color.Lerp(GlassBottom, GlassTop, k);
            if (y &gt;= h - 4)                                 // inner top light band
                c = new Color(c.r + 0.045f, c.g + 0.055f, c.b + 0.07f);
            for (int x = 0; x &lt; w; x++) t.SetPixel(x, y, new Color(c.r, c.g, c.b, GlassAlpha));
        }
        t.Apply();
        return t;
    }
    // white border ring, transparent center - tint via SpriteRenderer.color
    public static Texture2D MakeRimTex(int w, int h, int ringPx)
    {
        Texture2D t = NewTex(w, h, FilterMode.Point);
        for (int y = 0; y &lt; h; y++)
            for (int x = 0; x &lt; w; x++)
            {
                bool edge = x &lt; ringPx || x &gt;= w - ringPx || y &lt; ringPx || y &gt;= h - ringPx;
                t.SetPixel(x, y, edge ? new Color(1f, 1f, 1f, 0.95f) : new Color(1f, 1f, 1f, 0f));
            }
        t.Apply();
        return t;
    }
    // glow ramps UP toward the widget edge (peak where the rim sits) and
    // fades to zero at the quad edge - white mask, tint via color
    public static Texture2D MakeGlowTex(int w, int h, float rangePx)
    {
        Texture2D t = NewTex(w, h, FilterMode.Bilinear);
        float peak = rangePx &gt;= 12f ? 0.55f : 0.5f;
        for (int y = 0; y &lt; h; y++)
            for (int x = 0; x &lt; w; x++)
            {
                float dOuter = Mathf.Min(x, y, w - 1 - x, h - 1 - y);
                float a = peak * Mathf.Clamp01(dOuter / Mathf.Max(1f, rangePx));
                t.SetPixel(x, y, new Color(1f, 1f, 1f, a));
```

```
        }
    t.Apply();
    return t;
}
// top-left diagonal highlight streak (crystal specular), white mask
public static Texture2D MakeStreakTex(int w, int h)
{
    Texture2D t = NewTex(w, h, FilterMode.Bilinear);
    for (int y = 0; y &lt; h; y++)
        for (int x = 0; x &lt; w; x++)
        {
            float nx = w &gt; 1 ? (float)x / (w - 1) : 0f;
            float nty = h &gt; 1 ? (float)y / (h - 1) : 1f;   // 1 = top
            float a = Mathf.Max(0f, 1f - (nx + (1f - nty)) / 0.55f) * 0.5f;
            t.SetPixel(x, y, new Color(1f, 1f, 1f, a));
        }
    t.Apply();
    return t;
}
// bottom inner shadow (2D depth cue), white mask (tinted black)
public static Texture2D MakeShadowTex(int w, int h)
{
    Texture2D t = NewTex(w, h, FilterMode.Bilinear);
    for (int y = 0; y &lt; h; y++)
        for (int x = 0; x &lt; w; x++)
        {
            float ny = h &gt; 1 ? (float)y / (h - 1) : 0f;    // 0 = bottom
            float a = Mathf.Clamp01((0.72f - ny) / 0.28f) * 0.45f;
            t.SetPixel(x, y, new Color(1f, 1f, 1f, a));
        }
    t.Apply();
    return t;
}
// ---- internals ----
static int TexW(float wu) { return Mathf.Max(8, Mathf.RoundToInt(wu * PPU)); }
static int TexH(float hu) { return Mathf.Max(8, Mathf.RoundToInt(hu * PPU)); }
static int RimPx(Vector2 size) { return size.y &gt;= 2f ? 4 : 3; }
static float GlowRange(Vector2 size) { return size.y &gt;= 2f ? 0.45f : 0.3f; }
static Texture2D NewTex(int w, int h, FilterMode f)
{
    Texture2D t = new Texture2D(w, h, TextureFormat.RGBA32, false);
    t.filterMode = f;
    t.wrapMode = TextureWrapMode.Clamp;
    return t;
}
static Sprite TexturedSprite(Texture2D tex)
{
    return Sprite.Create(tex, new Rect(0, 0, tex.width, tex.height),
        new Vector2(0.5f, 0.5f), PPU);
}
```

```
        static SpriteRenderer Layer(Transform parent, string name, Sprite sprite, int order)
        {
            GameObject go = new GameObject(name);
            go.transform.SetParent(parent, false);
            SpriteRenderer sr = go.AddComponent&lt;SpriteRenderer&gt;();
            sr.sprite = sprite;
            sr.sortingOrder = order;
            return sr;
        }
        // r13 law: localScale is RELATIVE to sprite.bounds.size (px/PPU)
        static void Fit(SpriteRenderer sr, Vector2 worldSize)
        {
            Vector3 n = sr.sprite.bounds.size;
            sr.transform.localScale = new Vector3(worldSize.x / n.x, worldSize.y / n.y, 1f);
        }
    }
}
// VehicleRules.cs
// FluxVerse P-28(3) pool-coverage face (r44) + r93 vehicle systematization
// (P-69 slice-1): parked-vehicle pure rules (source
// pack = MiniGame S-library AA-016.02 modern exteriors, singles + animated
// tiers; S-library direct-use chain, license gate pre-cleared 2026-09-22,
// consumption provenance = TECH sec.9 P-21(3) r44/r45 + P-69 r93 rows).
// Street-life demand per DESIGN organ table: the roads layer is built and
// walked by robots (fleet avatars) and residents (census identities); parked
// cars / one bus / vendor carts / a bus-stop sign are static street furniture.
// v0 = STATIC parked scenery persisted in the scene (r35/r36 static-vs-transient
// boundary law); driving/parking animation is an M2 event-driven face, never a
// decorative loop. Frame files are tight-cropped by Tools/city/crop-vehicle-frames.ps1
// (the singles ship asymmetric transparent padding; tight crop = symmetric pivot
// so the ground-line math below is exact - r37 zero-floating law).
// r93 vehicle systematization (P-69 slice-1 face, R-20260924-m1-visual-fix
// sec.2-1 "elongated side views with wheels, no wheel-less placeholder
// blocks"): the three street sedans move from the singles tier (61x37 =
// the audited 1.65:1 "short and tall" defect) to the animated-strip 3/4
// frames (tight 78x36 incl. the pack's baked opaque ground-shadow band,
// 2.17:1, wheels verified); the bus moves to the animated sheet row sprite
// (115x62, pure side elevation, tires+hubcaps verified). The two vertical-
// avenue front/rear frames (32x57 = the "35px wheel-less block" reading at
// L0) are RETIRED from the scene - proper front/rear elevation frames are a
// documented later-acquisition debt, the strips only carry top-down
// verticals (view-type mismatch). Frame crops + the pixel-exact mirrored
// west sedan = Tools/city/crop-vehicle-frames.ps1 (r93 strip tier).
// Scale law: PPU24 divisor absorbs the pack's pixel density (r37 law); the
// divisor moves, the world law never does. Sizes: sedan 3.25x1.5u (3/4 tier
// incl. shadow band; body proper sits below the 1.33u resident line), bus
// 4.79x2.58u, camper 3.92x2.33u, carts 1.83x2.0u / 2.0x2.29u, stop sign
// 0.63x1.54u. Pack-internal length remains art-constrained (sedan 2.4 body
// lengths vs real 4-5, bus 3.6 vs 7 - honest note in TECH sec.9 r93 row;
// pack simply has no elongated sedan art - the r91 "65x20" survey estimate
```

```
// did not survive component-level measurement).
// Placement law: FEET ON THE GROUND LINE - Street class parks on the vertical
// middle of the 2-row road band (south street rows -8/-7 -&gt; ground y = -7,
// north street rows 7/8 -&gt; ground y = +8), so the feet cell is a Roads tile;
// Plaza class (carts, stop sign) stands on Ground pavement cells. The proof
// re-derives the feet cell from the live tilemaps, never from this comment.
// Clearance law: center distance &gt;= 2.6u to every robot, &gt;= 2.8u to every
// resident (r37 spacing family), &gt;= 2.6u between vehicles, and no overlap with
// any mounted neon-sign rect expanded by 0.4u (r35 law). Sorting: street layer
// order 7 = same as robots/residents (signs 6 &lt; street 7 &lt; tint 8). Five-color
// law untouched: vehicle paint is ambient scenery. ASCII. No 3D.
using UnityEngine;
namespace FluxVerse
{
    public static class VehicleRules
    {
        public const int Order = 7;            // street layer: signs 6 &lt; street 7 &lt; tint 8
        public const float PPU = 24f;           // density divisor (r37 law); importer enforced, never assumed
        public const int Count = 8;
        public const string NamePrefix = "Vehicle";
        public struct Veh
        {
            public string name;    // scene GO name (ASCII)
            public string path;    // project-relative asset path (Assets/...)
            public int pxW, pxH;   // tight-cropped sprite pixels
            public float x;        // world center x
            public float groundY;  // the ground line the wheels/feet stand on
            public bool street;   // true = parked on a Roads cell, false = Ground pavement
        }
        public static Veh At(int i) { return Table[i]; }
        public static string Name(int i) { return Table[i].name; }
        public static string Path(int i) { return Table[i].path; }
        public static float PxW(int i) { return Table[i].pxW; }
        public static float PxH(int i) { return Table[i].pxH; }
        public static float WorldW(int i) { return Table[i].pxW / PPU; }
        public static float WorldH(int i) { return Table[i].pxH / PPU; }
        public static Vector2 Pos(int i) { return new Vector2(Table[i].x, Table[i].groundY + WorldH(i) / 2f); }
        public static float FeetY(int i) { return Table[i].groundY; }
        public static bool IsStreet(int i) { return Table[i].street; }
        // framing: every vehicle fully inside the L0 view (ortho 20, aspect 16:9)
        // with margin, and inside the tint band y -16..+14 (an untinted sliver
        // would read as the r22 edge-band debt all over again)
        public static bool InView(int i, float halfW, float halfH)
        {
            Veh v = Table[i];
            float hw = WorldW(i) / 2f, hh = WorldH(i) / 2f;
            Vector2 c = Pos(i);
            return Mathf.Abs(c.x) + hw &lt;= halfW - 0.3f &amp;&amp; Mathf.Abs(c.y) + hh &lt;= halfH - 0.3f;
        }
        public static bool InTintBand(int i)
```

```
        {
            float top = Pos(i).y + WorldH(i) / 2f;
            return FeetY(i) &gt;= -16f + 0.1f &amp;&amp; top &lt;= 14f;
        }
        // the tile cell the feet point stands in (parking slot law)
        public static Vector2Int FeetCell(int i)
        {
            return new Vector2Int(Mathf.FloorToInt(Table[i].x), Mathf.FloorToInt(FeetY(i)));
        }
        static readonly Veh[] Table = new Veh[]
        {
            // 0. south street, west stretch in front of GAME - parked sedan (faces
            // west; r93: strip-tier 3/4 frame, pixel-exact mirror of the pack's
            // right-facing teal sedan); x sits clear of the r38 BIGGAME roof plate
            new Veh { name = "VehicleCarW",     path = "Assets/Art/Vehicles/frames/vehicle-car2-w.png",     pxW = 78,  p
            // 1. south street, east of the QUANT tower front - second color (faces east)
            new Veh { name = "VehicleCarQE",    path = "Assets/Art/Vehicles/frames/vehicle-car2-qe.png",    pxW = 78,  p
            // 2. south street, east stretch toward MEDIA - third color (faces east)
            new Veh { name = "VehicleCarE",     path = "Assets/Art/Vehicles/frames/vehicle-car2-e.png",     pxW = 78,  p
            // 3. GAME front plaza, far west edge - parked camper van (plaza class;
            //     r87 P-69 slice-3 move: the old south-street seat at (-30.5,-7)
            //     put the 2.33u-tall body over grass row -6 = the sec.8 "van
            //     crushing the greenbelt" r45 defect; plaza pavement = legal ground)
            new Veh { name = "VehicleCamperW",  path = "Assets/Art/Vehicles/frames/vehicle-camper-r.png",  pxW = 94,  px
            // 4. north street, east side - the bus (rows 7/8 band, ground line +8;
            //     r93: animated-sheet row sprite, pure side elevation facing east)
            new Veh { name = "VehicleBusN",     path = "Assets/Art/Vehicles/frames/vehicle-bus2-r.png",      pxW = 115,
            // 5. QUANT plaza, east edge - street-food cart (pavement class)
            new Veh { name = "VehicleCartQ",    path = "Assets/Art/Vehicles/frames/vehicle-cart-food.png",  pxW = 44,  p
            // 6. GAME front plaza, west edge - fruit/flower cart (pavement class)
            new Veh { name = "VehicleCartG",   path = "Assets/Art/Vehicles/frames/vehicle-cart-fruit.png", pxW = 48,  px
            // 7. north street, south sidewalk east of the bus - bus-stop sign
            // (pavement class; x clear of the north neon banner row, proof-caught)
            new Veh { name = "VehicleStopN",    path = "Assets/Art/Vehicles/frames/vehicle-stop-sign.png",  pxW = 15,  p
            // r93: indexes 8/9 (the two vertical-avenue front/rear sedans,
            // Car_Up_2 / Car_Down_4 singles 32x57) RETIRED - they read as the
            // audited "35px wheel-less placeholder block" at L0 and the pack
            // carries no replacement front/rear elevation frames (strip verticals
            // are top-down = view-type mismatch). Later-acquisition debt is
            // recorded in TECH sec.9 P-69 r93; proper avenue parking returns
            // with acquired frames, never with placeholder blocks.
        };
    }
}
// WaterFxRules.cs
// FluxVerse r155 (P-20260925-09 water &amp; daynight CEO order 09-25 ~18:00,
// spec R-20260925-water-daynight.md): pure rules table for the water-fx line
// (W1 frame cycle / W2 reflections / W3 foam / W4 shimmer / W5 rain law).
// Source law = Tools/city/waterfx-manifest.json (the r154 sandbox+asset bake,
// 27 assertions green; single geometry source - OfficeRules/LabsRules idiom,
```

```
// person/city data stays out per the r10 stale-copy law).
// Band law: builder live anchor cells x -50..50 rows -3..2 -&gt; 6 x 101 = 606
// painted cells; the SetTile sweep swaps VARIANTS only, never adds/removes
// cells (census identity). The saved scene keeps the builder's static paint
// hash as the boot state; the cycling law rewrites the full band on the
// adapter's first tick (manifest frame_cycle.first_tick).
// Frame law (W1): at water tick t, cell (x,y) shows frame
// ((x*31 + y*17 + t) mod 8) - the builder's spatial hash becomes the per-cell
// phase offset (31 mod 8 = 7 -&gt; neighbors mostly anti-phase, staggered
// wavefront). Fallback = mod 4 over the legacy registered set t_water_0/2/3/4
// (stays legal if a cycle tile ever fails to import).
// Tier law (W2 night must show): refl + shimmer alpha day/dawn 0, dusk 0.55,
// night 1.0; foam is the bank line - always 1.0. Dawn has no manifest row and
// maps to the day family (sunrise glow: no reflection band) - honest note in
// the TECH r155 row, proof gates the closed map.
// Wobble law (W5): x offset sequence [0, +1/16, -1/16, 0] world u per water
// tick (1 art px @ ppu16); rain doubles it. Foam stays pinned to the bank.
// Sibling render order (manifest sibling_order water &lt; refl &lt; shimmer &lt; foam):
// one sorting order 2 (above the Water tilemap 1, below street 7 / tint 8 -
// reflections receive the dusk/night tint like all city furniture; the Roads
// tilemap also sits at order 2, zero spatial overlap), distinct z so the
// same-order tie never falls to load order (r119 law): smaller z = on top.
// GAME_ANNEX reflection omitted v0 (spec W2 scope = three cities, manifest
// honest_notes). ASCII. No 3D.
using UnityEngine;
namespace FluxVerse
{
    public static class WaterFxRules
    {
        public const string Protocol = "fluxverse-waterfx/0.3";
        public const int BakedRound = 204;
        // ---- water band (builder live anchor) ----
        public const int CellX0 = -50, CellX1 = 50, RowY0 = -3, RowY1 = 2;
        public const int CensusBaseline = 606;   // (101 cells) x (6 rows)
        // ---- W1 frame cycle ----
        public const int FrameSlots = 8;          // t_water_0 .. t_water_7
        public const int FallbackFrames = 4;      // legacy registered set
        public const float TickSec = 0.5f;        // 2 fps
        public const float TickSecRain = 0.25f;   // 4 fps (W5)
        public static string FrameTileName(int slot) { return "t_water_" + slot; }
        public static int FrameSlotFor(int x, int y, int t)
        {
            return (((x * 31 + y * 17 + t) % FrameSlots) + FrameSlots) % FrameSlots;
        }
        // fallback: mod 4 over the registered vocabulary t_water_0/2/3/4
        public static int FallbackSlotFor(int x, int y, int t)
        {
            int f = (((x * 31 + y * 17 + t) % FallbackFrames) + FallbackFrames) % FallbackFrames;
            return f == 0 ? 0 : (f == 1 ? 2 : (f == 2 ? 3 : 4));
        }
```

```
// ---- tier alpha (W2; scope = refl + shimmer; foam = bank line) ----
public const float AlphaDay = 0f;
public const float AlphaDawn = 0f;      // day family (no manifest row - honest note)
public const float AlphaDusk = 0.55f;
public const float AlphaNight = 1f;
public const float FoamAlpha = 1f;
public static float TierAlpha(AmbientTier t)
{
    switch (t)
    {
        case AmbientTier.Dusk: return AlphaDusk;
        case AmbientTier.Night: return AlphaNight;
        default: return AlphaDay;   // Day + Dawn
    }
}
// ---- W3 foam ----
public const int FoamFrames = 4;         // foam frame = water tick mod 4
public static string FoamAssetPath(bool north, int frame)
{
    return "Assets/ArtPacks/water-fx/foam-" + (north ? "n" : "s") + "-" + frame + ".png";
}
// ---- r204 tile alpha (T-FV-122 S2, batch-2 order item 1
// "semi-transparent blue-purple") ----
// The r181 dusk rose wash is RETIRED: the r180 pre-registered rose
// parameter is overturned by the later CEO order text (r202 sec.6a
// compliant-change ruling); the r203 WaterTiles bake carries the
// authored blue-purple gradient, the wash family is removed and the
// live census returns to the 12 mount table (retirement face =
// waterfx-manifest v0.3 warm_wash_retired).
// The order's "semi-transparent" lands on the WATER TILEMAP renderer
// color: the r203 tiles bake opaque art, the sky quad behind shows
// through the band (dusk warm under-glow at the low rows = the water
// mirrors the sky, physically). Band [0.85, 0.95] pre-registered
// r202; value pinned 0.92 (sandbox: keeps the blue-purple saturation
// while the under-blend still reads). The r155-era light-blue tint
// (0.75, 0.85, 1) retired WITH the old navy art - tint neutral white.
public const float TileAlpha = 0.92f;
public const float TileAlphaMin = 0.85f, TileAlphaMax = 0.95f;
public static readonly Color TileTint = new Color(1f, 1f, 1f, TileAlpha);
// ---- W5 wobble ----
public const float WobbleStepU = 0.0625f;   // 1 art px @ ppu16
public static float WobbleOffset(int tick, bool rain)
{
    int i = ((tick % 4) + 4) % 4;
    float o = i == 0 ? 0f : (i == 1 ? WobbleStepU : (i == 2 ? -WobbleStepU : 0f));
    return rain ? o * 2f : o;
}
// ---- render family law ----
public const int SortOrder = 2;          // above Water tilemap (1), below street 7 / tint 8
public const float ReflZ = 0.3f;          // deepest (below shimmer, below foam)
```

```
public const float ShimmerZ = 0.2f;
public const float FoamZ = 0.1f;          // smallest z = on top (r119 tie law)
// ---- mount table (12 = 4 refl + 6 shimmer + 2 foam) ----
public const int MountCount = 12;
public const int ReflCount = 4, ShimCount = 6, FoamCount = 2;
public const int FamilyRefl = 0, FamilyShim = 1, FamilyFoam = 2;
public const string NamePrefix = "WaterFx";
public struct Mount
{
    public string name;             // scene GO name (ASCII)
    public string path;            // project-relative asset path
    public int pxW, pxH;           // tight sprite pixels
    public float ppu;              // importer tier
    public float x0, y0, x1, y1;   // world rect min/max corners
    public int family;             // Family* constant
    public string sign;            // shimmer: mounting sign GO name ("" otherwise)
    public int foamBase;            // foam: sprite index base (0 = north, 4 = south)
}
static readonly Mount[] Table = new Mount[]
{
    // W2 reflections - mirror semantics: the object edge nearest the
    // water lands on the bank line (roof crown at the bank edge).
    new Mount { name = "WaterFxReflTower", path = "Assets/ArtPacks/water-fx/refl-tower.png",
                pxW = 80, pxH = 48, ppu = 16f, x0 = -2f, y0 = 0f, x1 = 3f, y1 = 3f,
                family = FamilyRefl, sign = "", foamBase = 0 },
    new Mount { name = "WaterFxReflQuant", path = "Assets/ArtPacks/water-fx/facades/refl-quant.png",
                pxW = 120, pxH = 72, ppu = 24f, x0 = -2f, y0 = -3f, x1 = 3f, y1 = 0f,
                family = FamilyRefl, sign = "", foamBase = 0 },
    new Mount { name = "WaterFxReflGame", path = "Assets/ArtPacks/water-fx/facades/refl-game.png",
                pxW = 120, pxH = 72, ppu = 24f, x0 = -23f, y0 = -3f, x1 = -18f, y1 = 0f,
                family = FamilyRefl, sign = "", foamBase = 0 },
    new Mount { name = "WaterFxReflMedia", path = "Assets/ArtPacks/water-fx/facades/refl-media.png",
                pxW = 144, pxH = 72, ppu = 24f, x0 = 19f, y0 = -3f, x1 = 25f, y1 = 0f,
                family = FamilyRefl, sign = "", foamBase = 0 },
    // W4 shimmer - the outermost sign pair per city (overlap-free by
    // construction: 1.5u mounts centered on sign x, neighbor signs
    // &gt; 1.5u apart at the outer pair). Center plates NeonBigmoney /
    // NeonBiggame omitted - their glow is carried by the refl crown.
    new Mount { name = "WaterFxShimGameW", path = "Assets/ArtPacks/water-fx/shimmer-cyan.png",
                pxW = 24, pxH = 40, ppu = 16f, x0 = -23.25f, y0 = -3f, x1 = -21.75f, y1 = -0.5f,
                family = FamilyShim, sign = "NeonGameWest", foamBase = 0 },
    new Mount { name = "WaterFxShimGameE", path = "Assets/ArtPacks/water-fx/shimmer-cyan.png",
                pxW = 24, pxH = 40, ppu = 16f, x0 = -20.45f, y0 = -3f, x1 = -18.95f, y1 = -0.5f,
                family = FamilyShim, sign = "NeonGameWest2", foamBase = 0 },
    new Mount { name = "WaterFxShimQuantW", path = "Assets/ArtPacks/water-fx/shimmer-gold.png",
                pxW = 24, pxH = 40, ppu = 16f, x0 = -2.2f, y0 = -3f, x1 = -0.7f, y1 = -0.5f,
                family = FamilyShim, sign = "NeonQuantL", foamBase = 0 },
    new Mount { name = "WaterFxShimQuantE", path = "Assets/ArtPacks/water-fx/shimmer-gold.png",
                pxW = 24, pxH = 40, ppu = 16f, x0 = 0.7f, y0 = -3f, x1 = 2.2f, y1 = -0.5f,
                family = FamilyShim, sign = "NeonQuantR", foamBase = 0 },
```

```
        new Mount { name = "WaterFxShimMediaE", path = "Assets/ArtPacks/water-fx/shimmer-magenta.png",
                    pxW = 24, pxH = 40, ppu = 16f, x0 = 19.45f, y0 = -3f, x1 = 20.95f, y1 = -0.5f,
                    family = FamilyShim, sign = "NeonMediaEast", foamBase = 0 },
        new Mount { name = "WaterFxShimMediaR", path = "Assets/ArtPacks/water-fx/shimmer-magenta.png",
                    pxW = 24, pxH = 40, ppu = 16f, x0 = 21.65f, y0 = -3f, x1 = 23.15f, y1 = -0.5f,
                    family = FamilyShim, sign = "NeonMediaRoof", foamBase = 0 },
        // W3 foam - 1px bank line + sparse bubble row (embankment gradient
        // v0), 4 frames synced to the water tick; foam is bank-pinned.
        new Mount { name = "WaterFxFoamN", path = "Assets/ArtPacks/water-fx/foam-n-0.png",
                    pxW = 1600, pxH = 2, ppu = 16f, x0 = -50f, y0 = 2.875f, x1 = 50f, y1 = 3f,
                    family = FamilyFoam, sign = "", foamBase = 0 },
        new Mount { name = "WaterFxFoamS", path = "Assets/ArtPacks/water-fx/foam-s-0.png",
                    pxW = 1600, pxH = 2, ppu = 16f, x0 = -50f, y0 = -3f, x1 = 50f, y1 = -2.875f,
                    family = FamilyFoam, sign = "", foamBase = 4 },
    };
    public static Mount At(int i) { return Table[i]; }
    public static string Name(int i) { return Table[i].name; }
    public static string Path(int i) { return Table[i].path; }
    public static int PxW(int i) { return Table[i].pxW; }
    public static int PxH(int i) { return Table[i].pxH; }
    public static int FamilyOf(int i) { return Table[i].family; }
    public static string SignOf(int i) { return Table[i].sign; }
    public static int FoamBase(int i) { return Table[i].foamBase; }
    public static float X0(int i) { return Table[i].x0; }
    public static float Y0(int i) { return Table[i].y0; }
    public static float X1(int i) { return Table[i].x1; }
    public static float Y1(int i) { return Table[i].y1; }
    public static float WorldW(int i) { return Table[i].x1 - Table[i].x0; }
    public static float WorldH(int i) { return Table[i].y1 - Table[i].y0; }
    public static float CenterX(int i) { return (Table[i].x0 + Table[i].x1) * 0.5f; }
    public static float CenterY(int i) { return (Table[i].y0 + Table[i].y1) * 0.5f; }
    public static float ZOf(int i)
    {
        return Table[i].family == FamilyRefl ? ReflZ
             : (Table[i].family == FamilyShim ? ShimmerZ : FoamZ);
    }
    // natural-size law: sprite world size = px / ppu == the rect (scale 1)
    public static bool NaturalSizeMatches(int i, Sprite s)
    {
        float w = s.rect.width / Table[i].ppu, h = s.rect.height / Table[i].ppu;
        return Mathf.Abs(w - WorldW(i)) &lt; 1e-4f &amp;&amp; Mathf.Abs(h - WorldH(i)) &lt; 1e-4f;
    }
    // index finders (single-family invariants asserted by the proof)
    public static int FoamIndexOf(bool north)
    {
        for (int i = 0; i &lt; MountCount; i++)
            if (Table[i].family == FamilyFoam &amp;&amp; (north ? Table[i].y0 &gt; 0f : Table[i].y0 &lt; 0f)) return
        return -1;
    }
}
```

```
}
// WindowLightRules.cs
// FluxVerse r160 (P-20260925-09 D2 window-light heat, CEO order 09-25 ~18:00,
// spec R-20260925-water-daynight.md sec2 item2): pure rules table for the
// window-light line. Source law = Tools/city/windowlight-manifest.json (the
// r159 sandbox, 48 checks green - single parameter source; OfficeRules /
// LabsRules idiom). Night window lit-rate = each company's LIVE activity from
// world-state zones[] (scan floors gaming 0.5 / media 0.4 upstream; live
// values observed 1.0) - ZERO static baked numbers (stale honesty law).
//
// WINDOW LAW (r159): every cell's face type is the Block() paving law -
// top row = roof (zero windows), walls alternate wallA/wallB by
// (cell_x + cell_y) &amp;amp; 1. gray_c = ART windows (glass px 77,146,239 in
// GuttyKreum_CleanCity_074: two 2x8 windows x7/x10, band y5..12, mullion
// rows y6-7 inside); gray_a = the eave tile (zero glass px, sandbox-proven)
// so windows are RHYTHM-DRAWN at the exact gray_c positions (one city-wide
// window rhythm law); gray_b = plank wall with 1px vertical slits on the
// plank-face centers x2/6/10/14 (seams x0/4/8/12, sandbox-proven), same band.
//
// LIT LAW v2 (r181, duskgold-manifest.windowlight_rate_v2 - the r180 S6a
// parameter source; the v1 face lit 100 pct of windows at the live activity
// 1.0 = the art-spec "all-lit windows are fake" violation, LIVE at census):
// lit iff fnv1a32(bid|date|widx) mod 10000 &amp;lt; floor(effective_rate*10000+0.5)
// where effective_rate = BASE_RATE(0.30) * Clamp01(zone activity) *
// floor_factor. floor_factor = linear per wall row bottom-&amp;gt;top 1.0-&amp;gt;0.6
// (denser low, sparser high - the art-target window rhythm); wallRows==1 -&amp;gt;
// factor 1.0. ThresholdFor / Fnv / widx enumeration law UNCHANGED (r159);
// the adapter multiplies BASE_RATE * factor at the single LitAt call site.
// Band check (r180 PS sweep, proof re-pins): overall lit fraction at
// activity 1.0 lands in [0.20, 0.30] every calendar date (5-date pinned
// 0.2305..0.2605). date = the Beijing calendar date (day-stable, cross-day
// variety, same-input-same-output per day - draw.py family). widx = window
// scan order: row-major cells (cy outer, cx inner) then family rect order.
// Zone rate: QUANT-&amp;gt;quant, GAME_MAIN+ANNEX-&amp;gt;gaming, MEDIA-&amp;gt;media; the four
// north historical facades = 'city' = round(mean(quant,gaming,media), 2)
// (the strip belongs to no single company). Excluded by design: brain tower
// (superbody-blue CEO family), office band (v1 candidate), labs (glow).
//
// RENDER: order 3 with z -0.5 = the same-order z-toward-camera trick (r155
// water / r158 light-fx family): renders ABOVE the order-3 city tilemaps at
// z 0, below props 4 / signs 6 / street 7 / tint 8. Tier alpha closed set
// day/dawn 0, dusk 0.55, night 1.0 (r155 water family). Colors bake into the
// runtime texture (fill warm yellow / core bright interior, five-color law
// untouched - human windows are the warm-yellow family, city-core sec.VI);
// the sprite color stays neutral white carrying only the tier alpha.
// ASCII. No 3D.
using System;
using UnityEngine;
namespace FluxVerse
{
```

```
public static class WindowLightRules
{
    public const string Protocol = "fluxverse-windowlight/0.2";
    public const int BakedRound = 181;
    public const string MountPrefix = "WindowLight";
    public const int Order = 3;
    public const float Z = -0.5f;
    public const float Ppu = 16f;
    public const float PollIntervalSec = 10f;   // CityAmbient polling law
    // ---- tier alpha (manifest laws.tier_alpha) ----
    public static float AlphaFor(AmbientTier t)
    {
        if (t == AmbientTier.Dusk) return 0.55f;
        if (t == AmbientTier.Night) return 1.0f;
        return 0f;   // day + dawn: human windows are an evening phenomenon
    }
    // ---- colors (manifest laws.fill_rgb / core_rgb) ----
    public static readonly Color Fill = new Color(255f / 255f, 212f / 255f, 130f / 255f, 1f);
    public static readonly Color Core = new Color(255f / 255f, 236f / 255f, 180f / 255f, 1f);
    public const int CoreMinH = 6;   // h &gt;= 6 windows get the middle 2 rows in core
    // ---- window families (manifest window_families, art px in the 16px cell) ----
    public struct WinRect { public int ax, ay, w, h; }
    static readonly WinRect[] FamGrayC = new WinRect[]
    {
        new WinRect { ax = 7, ay = 5, w = 2, h = 8 },
        new WinRect { ax = 10, ay = 5, w = 2, h = 8 }
    };
    // gray_a rhythm law: identical positions to the gray_c ART windows
    static readonly WinRect[] FamGrayA = FamGrayC;
    static readonly WinRect[] FamGrayB = new WinRect[]
    {
        new WinRect { ax = 2, ay = 5, w = 1, h = 8 },
        new WinRect { ax = 6, ay = 5, w = 1, h = 8 },
        new WinRect { ax = 10, ay = 5, w = 1, h = 8 },
        new WinRect { ax = 14, ay = 5, w = 1, h = 8 }
    };
    public const string FamGrayAName = "t_wall_gray_a";
    public const string FamGrayBName = "t_wall_gray_b";
    public const string FamGrayCName = "t_wall_gray_c";
    static WinRect[] FamilyFor(string tile)
    {
        if (tile == FamGrayBName) return FamGrayB;
        if (tile == FamGrayCName) return FamGrayC;
        return FamGrayA;   // gray_a + anything unknown degrades to the rhythm
    }
    // ---- buildings (manifest buildings[]; rect = world [x0,y0,x1,y1]) ----
    public struct Building
    {
        public string id; public string zone;
        public int cx0, cx1;           // cells_x span (inclusive)
```

```
    public int yBase, rows;
    public string wallA, wallB, roof;
    public float x0, y0, x1, y1;
    public int expectedWindows;
}
static readonly Building[] Table = new Building[]
{
    new Building { id = "WLQuant", zone = "quant", cx0 = -2, cx1 = 2,
        yBase = -16, rows = 8, wallA = FamGrayBName, wallB = FamGrayCName,
        roof = "t_roof_a", x0 = -2f, y0 = -16f, x1 = 3f, y1 = -8f, expectedWindows = 106 },
    new Building { id = "WLGameMain", zone = "gaming", cx0 = -23, cx1 = -19,
        yBase = -16, rows = 5, wallA = FamGrayAName, wallB = FamGrayAName,
        roof = "t_roof_b", x0 = -23f, y0 = -16f, x1 = -18f, y1 = -11f, expectedWindows = 40 },
    new Building { id = "WLGameAnnex", zone = "gaming", cx0 = -26, cx1 = -25,
        yBase = -16, rows = 3, wallA = FamGrayAName, wallB = FamGrayAName,
        roof = "t_roof_b", x0 = -26f, y0 = -16f, x1 = -24f, y1 = -13f, expectedWindows = 8 },
    new Building { id = "WLMedia", zone = "media", cx0 = 19, cx1 = 24,
        yBase = -14, rows = 6, wallA = FamGrayBName, wallB = FamGrayBName,
        roof = "t_roof_b", x0 = 19f, y0 = -14f, x1 = 25f, y1 = -8f, expectedWindows = 120 },
    new Building { id = "WLNw", zone = "city", cx0 = -28, cx1 = -26,
        yBase = 9, rows = 3, wallA = FamGrayAName, wallB = FamGrayBName,
        roof = "t_roof_b", x0 = -28f, y0 = 9f, x1 = -25f, y1 = 12f, expectedWindows = 18 },
    new Building { id = "WLNwMid", zone = "city", cx0 = -9, cx1 = -7,
        yBase = 9, rows = 3, wallA = FamGrayCName, wallB = FamGrayAName,
        roof = "t_roof_b", x0 = -9f, y0 = 9f, x1 = -6f, y1 = 12f, expectedWindows = 12 },
    new Building { id = "WLNeMid", zone = "city", cx0 = 7, cx1 = 9,
        yBase = 9, rows = 3, wallA = FamGrayBName, wallB = FamGrayCName,
        roof = "t_roof_b", x0 = 7f, y0 = 9f, x1 = 10f, y1 = 12f, expectedWindows = 18 },
    new Building { id = "WLNe", zone = "city", cx0 = 26, cx1 = 28,
        yBase = 9, rows = 3, wallA = FamGrayAName, wallB = FamGrayCName,
        roof = "t_roof_b", x0 = 26f, y0 = 9f, x1 = 29f, y1 = 12f, expectedWindows = 12 }
};
public const int Count = 8;
public const int TotalWindows = 334;   // 106+40+8+120+18+12+18+12 (proof re-derives)
public static Building At(int i) { return Table[i]; }
public static int IndexOf(string id)
{
    for (int i = 0; i &lt; Count; i++) if (Table[i].id == id) return i;
    return -1;
}
public static string MountName(int i) { return MountPrefix + Table[i].id; }
public static float CenterX(int i) { return (Table[i].x0 + Table[i].x1) * 0.5f; }
public static float CenterY(int i) { return (Table[i].y0 + Table[i].y1) * 0.5f; }
public static int PxW(int i) { return (int)Math.Floor((Table[i].x1 - Table[i].x0) * 16f + 0.5f); }
public static int PxH(int i) { return (int)Math.Floor((Table[i].y1 - Table[i].y0) * 16f + 0.5f); }
// ---- window enumeration (widx law: row-major cells, family rect order) ----
public struct Window
{
    public int cx, cy;        // cell (world cell coords)
    public int ax, ay, w, h;   // art px inside the 16px cell (ay from tile top)
```

```
}
// enumerates every window of building i in the canonical widx order
public static Window[] WindowsOf(int i)
{
    Building b = Table[i];
    WinRect[] fa = FamilyFor(b.wallA), fb = FamilyFor(b.wallB);
    int wallRows = b.rows - 1;             // top row = roof, zero windows
    int cells = (b.cx1 - b.cx0 + 1) * wallRows;
    int total = 0;
    for (int cy = b.yBase; cy &lt;= b.yBase + b.rows - 2; cy++)
        for (int cx = b.cx0; cx &lt;= b.cx1; cx++)
            total += (((cx + cy) &amp; 1) == 0) ? fa.Length : fb.Length;
    Window[] list = new Window[total];
    int k = 0;
    for (int cy = b.yBase; cy &lt;= b.yBase + b.rows - 2; cy++)
        for (int cx = b.cx0; cx &lt;= b.cx1; cx++)
        {
            WinRect[] fam = (((cx + cy) &amp; 1) == 0) ? fa : fb;
            for (int r = 0; r &lt; fam.Length; r++)
            {
                Window w = new Window();
                w.cx = cx; w.cy = cy;
                w.ax = fam[r].ax; w.ay = fam[r].ay; w.w = fam[r].w; w.h = fam[r].h;
                list[k++] = w;
            }
        }
    return list;
}
public static int WindowCountOf(int i) { return WindowsOf(i).Length; }
// window world rect (manifest rect_math)
public static void WorldRectOf(Building b, Window w,
    out float wx0, out float wy0, out float wx1, out float wy1)
{
    wx0 = w.cx + w.ax / 16f;
    wx1 = w.cx + (w.ax + w.w) / 16f;
    wy1 = (w.cy + 1) - w.ay / 16f;                 // top
    wy0 = (w.cy + 1) - (w.ay + w.h) / 16f;         // bottom
}
// ---- FNV-1a 32 (standard; golden vectors pin the C# build) ----
public static uint Fnv1a(string s)
{
    uint h = 2166136261u;
    for (int i = 0; i &lt; s.Length; i++)
    {
        h ^= (uint)(s[i] &amp; 0xFF);   // ASCII domain
        h *= 16777619u;
    }
    return h;
}
public static uint ThresholdFor(float rate)
```

```
        { return (uint)Math.Floor(rate * 10000.0 + 0.5); }   // round-half-up (r157 law)
        public static bool LitAt(string bid, string date, int widx, float rate)
        {
            return Fnv1a(bid + "|" + date + "|" + widx) % 10000u &lt; ThresholdFor(rate);
        }
        // ---- rate law v2 (duskgold-manifest.windowlight_rate_v2, r180/r181) ----
        public const float BaseRate = 0.30f;   // law constant; activity stays LIVE
        public static float FloorFactor(Building b, Window w)
        {
            int wallRows = b.rows - 1;                 // top row = roof, zero windows
            if (wallRows &lt;= 1) return 1f;
            float rowFromBottom = w.cy - b.yBase;      // 0 = bottom wall row
            return 1f - 0.4f * (rowFromBottom / (wallRows - 1));
        }
        // the single law entry the adapter feeds LitAt (v2: rate is EFFECTIVE)
        public static float EffectiveRate(Building b, Window w, float zoneRate)
        {
            return BaseRate * Clamp01(zoneRate) * FloorFactor(b, w);
        }
        // ---- zone rates (manifest zone_rate law) ----
        public static float Round2(float v) { return (float)Math.Round(v, 2); }
        public static float ZoneRateFor(string zone, float quant, float gaming, float media)
        {
            if (zone == "quant") return Round2(quant);
            if (zone == "gaming") return Round2(gaming);
            if (zone == "media") return Round2(media);
            return Round2((quant + gaming + media) / 3f);   // north 'city' mean
        }
        public static float Clamp01(float v) { return v &lt; 0f ? 0f : (v &gt; 1f ? 1f : v); }
        // ---- texture px mapping (building sprite, y=0 bottom row) ----
        // cell (cx,cy) block: tx0 = (cx - b.cx0) * 16, rows (cy - yBase) * 16 ..
        // window art rows ay..ay+h-1 (top-down) map to bottom-up rows
        // blockBottom + (16 - ay - h) .. blockBottom + (15 - ay)
        public static void TexRectOf(Building b, Window w,
            out int tx0, out int ty0, out int tw, out int th)
        {
            tw = w.w; th = w.h;
            tx0 = (w.cx - b.cx0) * 16 + w.ax;
            int blockBottom = (w.cy - b.yBase) * 16;
            ty0 = blockBottom + (16 - w.ay - w.h);
        }
        // core-row law: h &gt;= 6 windows carry the middle 2 rows in core color
        public static bool IsCoreRow(int h, int rowFromTop)
        {
            if (h &lt; CoreMinH) return false;
            int start = (h - 2) / 2;
            return rowFromTop &gt;= start &amp;&amp; rowFromTop &lt; start + 2;
        }
    }
}
```